

AV - repetition - no_frac - Serial position analysis

```
# do this in calling R script
# library(ggplot2)
# library(fmsb) # for TestModels
# library(lme4)
# library(kableExtra)
# library(MASS) # for dropterm
# library(tidyverse)
# library(dominanceanalysis)
# library(cowplot) # for multiple plots in one figure
# library(formatters) # to wrap strings for plot titles
# library(pals) # for continuous color palettes

# load needed functions
# source(paste0(RootDir, "/src/function_library/sp_functions.R"))
# do this in the calling R script

# for testing
# CurPat <- "GM"
# CurTask <- "repetition"
# MinLength <- 4
# MaxLength <- 10
# BestModelIndexL1 <- 1
# BestModelIndexL2 <- 1
# BestModelIndexL3 <- 1
# ReportTitle <- paste(CurPat, " - ", CurTask, " - ", RunLabel, " - Serial position analysis")
# RandomSamples <- 10
# DoSimulations <- FALSE
# RemoveFracPres <- TRUE

CurPat <- params$CurPat
CurTask <- params$CurTask
MinLength <- params$MinLength
MaxLength <- params$MaxLength
BestModelIndexL1 <- params$BestModelIndexL1
BestModelIndexL2 <- params$BestModelIndexL2
BestModelIndexL3 <- params$BestModelIndexL3
ReportTitle <- params$ReportTitle
RandomSamples <- params$RandomSamples
DoSimulations <- params$DoSimulations
RemoveFracPres <- params$RemoveFracPres

# done in calling script
# print(paste0("Using root dir: ", RootDir))
# RunDir <- paste0(paste0(RootDir, "/output/"), RunLabel))
# if(!dir.exists(RunDir)){
#   dir.create(RunDir, showWarnings = TRUE)
#   print(paste0("created run directory: ", RunDir))
# }
```

```

# # create patient directory if it doesn't already exist
# PatientDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat)
# if(!dir.exists(PatientDir)){
#   dir.create(PatientDir, showWarnings = TRUE)
#   print(paste0("created participant directory: ", PatientDir))
# }
# TablesDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/tables/")
# if(!dir.exists(TablesDir)){
#   dir.create(TablesDir, showWarnings = TRUE)
#   print(paste0("created tables directory: ", TablesDir))
# }
# FigDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/fig/")
# if(!dir.exists(FigDir)){
#   dir.create(FigDir, showWarnings = TRUE)
#   print(paste0("created figures directory: ", FigDir))
# }
# ReportsDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/reports/")
# if(!dir.exists(ReportsDir)){
#   dir.create(ReportsDir, showWarnings = TRUE)
#   print(paste0("created reports directory: ", ReportsDir))
# }
results_report_DF <- data.frame(Description=character(), ParameterValue=double())

ModelDatFilename<-paste0(RootDir, "/data/", CurPat, "/", CurPat, "_", CurTask, "_posdat.csv")
if(!file.exists(ModelDatFilename)){
  print("**ERROR** Input data file not found")
  print(sprintf("\tfile: ", ModelDatFilename))
  print(sprintf("\troot dir: ", RootDir))
}
PosDat<-read.csv(ModelDatFilename)

# may already be done in datafile
# if(RemoveFinalPosition){
#   PosDat <- PosDat[PosDat$pos < PosDat$stimlen,] # remove final position
# }
PosDat <- PosDat[PosDat$stimlen >= MinLength,] # include only lengths with sufficient N
PosDat <- PosDat[PosDat$stimlen <= MaxLength,]
# include only correct and nonword errors (not no response, word or w+nw errors)
PosDat <- PosDat[(PosDat$err_type == "correct") | (PosDat$err_type == "nonword"), ]
PosDat <- PosDat[(PosDat$pos) != (PosDat$stimlen), ]
# make sure final positions have been removed
PosDat$stim_number <- NumberStimuli(PosDat)
PosDat$raw_log_freq <- PosDat$log_freq
PosDat$log_freq <- PosDat$log_freq - mean(PosDat$log_freq) # centre frequency
OrigDataLen <- nrow(PosDat)
print(sprintf ("Original data has %d values", OrigDataLen))

## [1] "Original data has 4307 values"

if(RemoveFracPres){ # eliminate fractional preserved values?
  PosDat <- PosDat[(PosDat$preserved == 0) | (PosDat$preserved == 1),]
  DataLen<- nrow(PosDat)
  print(sprintf("Data without fractional preserved values has %d values", DataLen))
  print(sprintf("%d values eliminated.", OrigDataLen - DataLen))
}

```

```
## [1] "Data without fractional preserved values has 4272 values"
## [1] "35 values eliminated."
```

```
PosDat$CumPres <- CalcCumPres(PosDat)
PosDat$CumErr <- CalcCumErrFromPreserved(PosDat)
```

```
# plot distribution of complex onsets and codas
# across serial position in the stimuli -- do complexities concentrate
# on one part of the serial position curve?
# syll_component = 1 is coda
# syll_component = S is satellite
```

```
# get counts of syllable components in different serial positions
```

```
syll_comp_dist_N <- PosDat %>% mutate(pos_factor = as.factor(pos), syll_component_factor = as.factor(syll_component))
```

```
syll_comp_dist_perc <- syll_comp_dist_N %>% ungroup() %>%
  mutate(across(O:S, ~ . / total))
```

```
kable(syll_comp_dist_N)
```

pos_factor	O	P	V	1	S	total
1	535	33	126	NA	NA	694
2	64	NA	422	98	109	693
3	311	NA	165	202	15	693
4	292	NA	239	66	37	634
5	231	NA	203	72	37	543
6	202	1	137	68	22	430
7	174	NA	102	29	19	324
8	93	NA	55	25	4	177
9	75	NA	2	NA	7	84

```
kable(syll_comp_dist_perc)
```

pos_factor	O	P	V	1	S	total
1	0.7708934	0.0475504	0.1815562	NA	NA	694
2	0.0923521	NA	0.6089466	0.1414141	0.1572872	693
3	0.4487734	NA	0.2380952	0.2914863	0.0216450	693
4	0.4605678	NA	0.3769716	0.1041009	0.0583596	634
5	0.4254144	NA	0.3738490	0.1325967	0.0681400	543
6	0.4697674	0.0023256	0.3186047	0.1581395	0.0511628	430
7	0.5370370	NA	0.3148148	0.0895062	0.0586420	324
8	0.5254237	NA	0.3107345	0.1412429	0.0225989	177
9	0.8928571	NA	0.0238095	NA	0.0833333	84

```
# get percentages only from summary table
```

```
syll_comp_perc <- syll_comp_dist_perc[,2:(ncol(syll_comp_dist_perc)-1)]
```

```
syll_comp_perc$pos <- as.integer(as.character(syll_comp_dist_perc$pos_factor))
```

```
syll_comp_perc <- syll_comp_perc %>% rename("Coda" = "1", "Satellite" = "S")
```

```
# pivot to long format for plotting
```

```
syll_comp_plot_data <- syll_comp_perc %>% pivot_longer(cols=seq(1:(ncol(syll_comp_perc)-1)),
  names_to="syll_component", values_to="percent") %>% filter(syll_component %in% c("Coda", "Satellite"))
```

```
# plot satellite and coda occurrences across position
```

```
syll_comp_plot <- ggplot(syll_comp_plot_data,
  aes(x=pos,y=percent,group=syll_component,
    color=syll_component,
    linetype = syll_component,
    shape = syll_component)) +
  geom_line() + geom_point() + scale_color_manual(name="Syllable component", values = palette_values) +
syll_comp_plot <- syll_comp_plot + scale_y_continuous(name="Percent of segment types") + scale_linetype_manual(name="Syllable component", values = palette_linetypes) +
  scale_shape_discrete(name="Syllable component")
ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask, "_pos_of_syll_complexities_in_stimuli.png"), plot=syll_comp_plot)
```

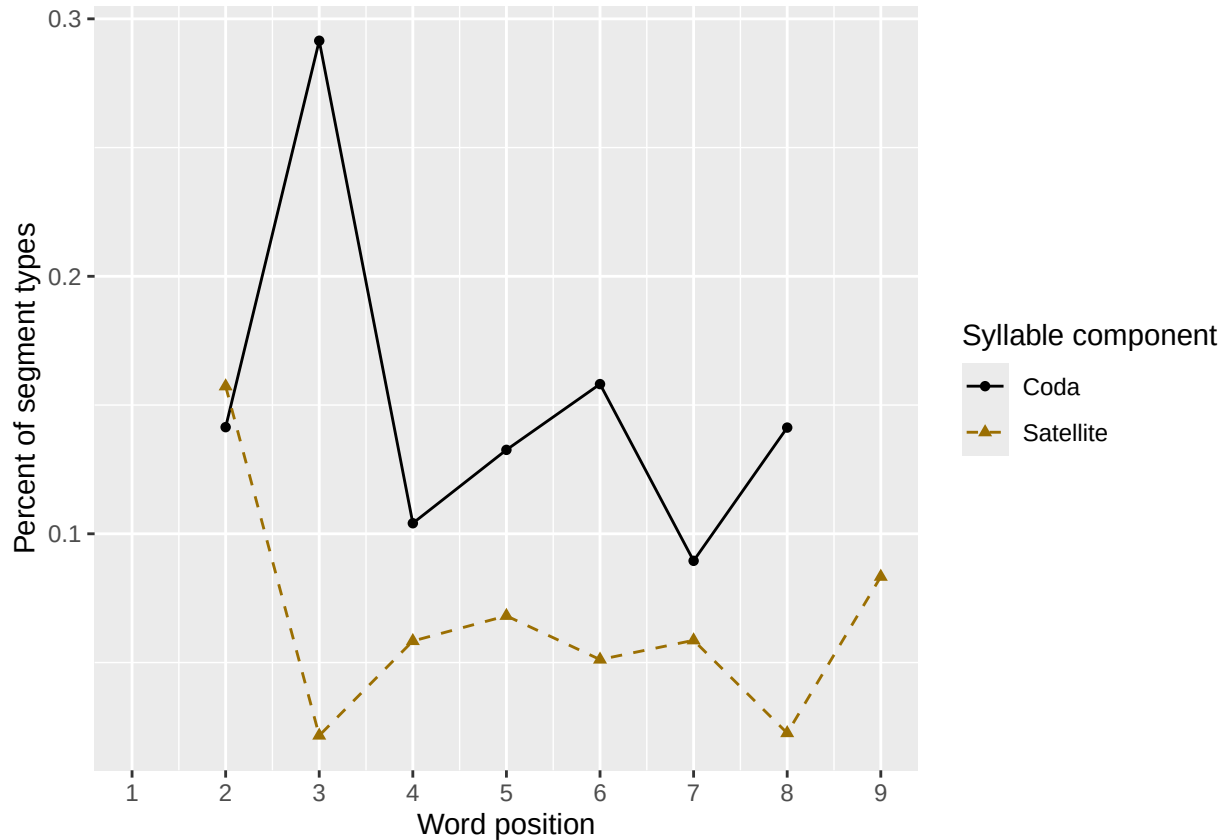
```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_line()`).
```

```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_point()`).
```

```
syll_comp_plot
```

```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_line()`).
```

```
## Removed 3 rows containing missing values or values outside the scale range (`geom_point()`).
```



```
# len/pos table
```

```
pos_len_summary <- PosDat %>% group_by(stimlen,pos) %>% summarise(preserved = mean(preserved))
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
min_preserved_raw <- min(as.numeric(unlist(pos_len_summary$preserved)))
```

```
max_preserved_raw <- max(as.numeric(unlist(pos_len_summary$preserved)))
```

```
preserved_range <- (max_preserved_raw - min_preserved_raw)
```

```

min_preserved <- min_preserved_raw - (0.1 * preserved_range)
max_preserved <- max_preserved_raw + (0.1 * preserved_range)
pos_len_table <- pos_len_summary %>% pivot_wider(names_from = pos, values_from = preserved)
write.csv(pos_len_table, paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask, "_preserved_percentages_by_len",
pos_len_table

```

```

## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1      4 0.814 0.915 0.932 NA      NA      NA      NA      NA      NA
## 2      5 0.819 0.915 0.925 0.892 NA      NA      NA      NA      NA
## 3      6 0.735 0.842 0.947 0.947 0.885 NA      NA      NA      NA
## 4      7 0.685 0.869 0.887 0.870 0.963 0.926 NA      NA      NA
## 5      8 0.703 0.836 0.829 0.890 0.932 0.946 0.946 NA      NA
## 6      9 0.607 0.761 0.793 0.912 0.946 0.879 0.957 0.968 NA
## 7     10 0.639 0.778 0.880 0.952 0.929 0.917 0.929 0.964 0.869

```

```

# len/pos table
pos_len_N <- PosDat %>% group_by(stimlen, pos) %>% summarise(preserved = mean(preserved), N=n()) %>% dplyr::

```

`summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

```

pos_len_N_table <- pos_len_N %>% pivot_wider(names_from = pos, values_from = N)
write.csv(pos_len_N_table, paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask,
"_N_by_len_position.csv"))
pos_len_N_table

```

```

## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <int> <int> <int> <int> <int> <int> <int> <int> <int>
## 1      4    59    59    59    NA    NA    NA    NA    NA    NA
## 2      5    94    94    93    93    NA    NA    NA    NA    NA
## 3      6   113   114   114   114   113    NA    NA    NA    NA
## 4      7   108   107   106   108   107   108    NA    NA    NA
## 5      8   148   146   146   145   147   147   147    NA    NA
## 6      9    89    92    92    91    92    91    93    93    NA
## 7     10    83    81    83    83    84    84    84    84    84

```

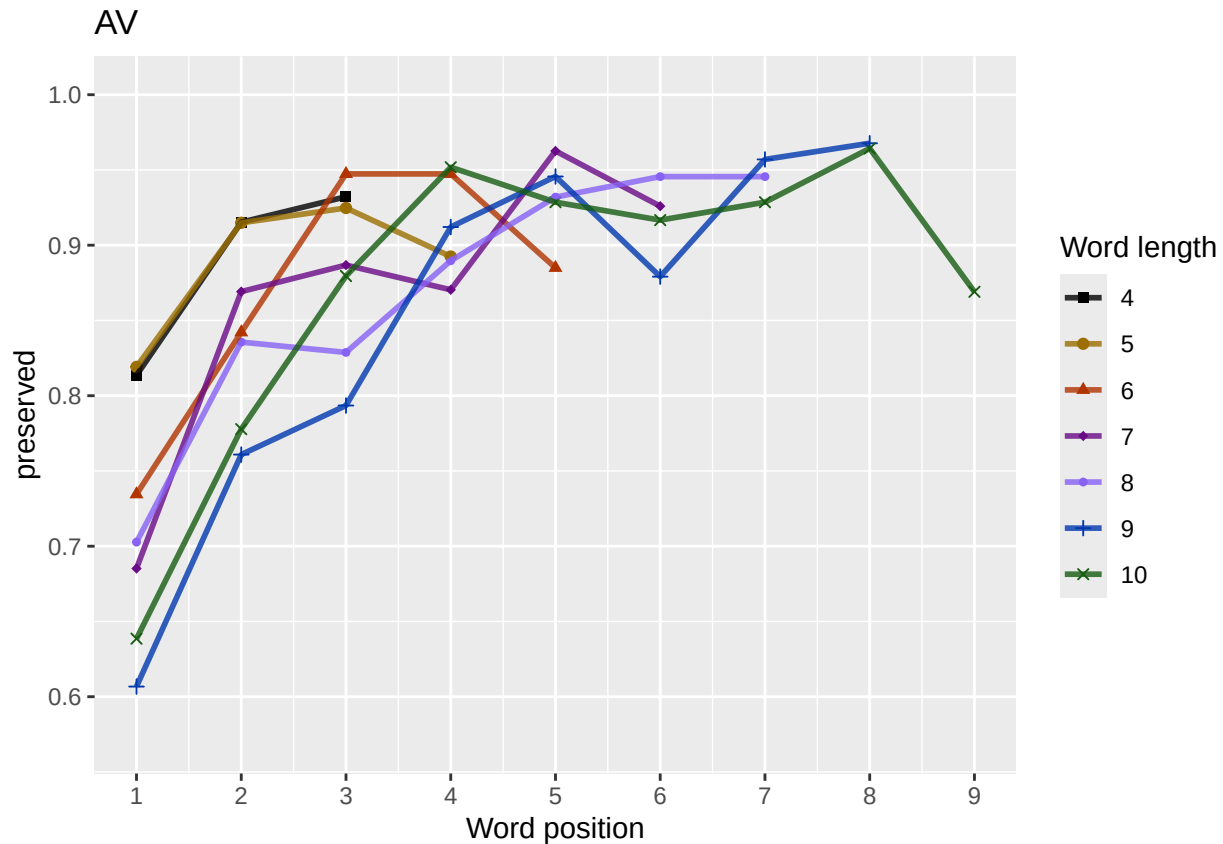
```

obs_linetypes <- c("solid", "solid", "solid", "solid",
"solid", "solid", "solid", "solid", "solid")
pred_linetypes <- "dashed"
pos_len_summary$stimlen <- factor(pos_len_summary$stimlen)
pos_len_summary$pos <- factor(pos_len_summary$pos)
# len_pos_plot <- ggplot(pos_len_summary, aes(x=pos, y=preserved, group=stimlen, shape=stimlen, color=stimlen))
len_pos_plot <- plot_len_pos_obs_predicted(PosDat,
paste0(CurPat),
NULL,
c(min_preserved, max_preserved),
palette_values,
shape_values,
obs_linetypes,
pred_linetypes)

```

`summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask,
              "_percent_preserved_by_length_pos.png"),
       plot=len_pos_plot, device="png", unit="cm", width=15, height=11)
len_pos_plot
```



Length and position

```
# length and position
```

```
LPMModelEquations<-c("preserved ~ 1",
  "preserved ~ stimlen",
  "preserved ~ pos",
  "preserved ~ stimlen + pos",
  "preserved ~ stimlen*pos",
  "preserved ~ I(pos^2)+pos",
  "preserved ~ stimlen + I(pos^2) + pos",
  "preserved ~ stimlen * (I(pos^2) + pos)"
)
```

```
LPres<-TestModels(LPMModelEquations,PosDat)
```

```
## *****
## model index: 8
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
```

```

##
## Coefficients:
##      (Intercept)      stimlen      I(pos^2)      pos  stimlen:I(pos^2)      stimlen:I(pos^2)
##      2.069824      -0.257112      -0.055491      0.425117      -0.001511      0.001511
##
## Degrees of Freedom: 4271 Total (i.e. Null); 4266 Residual
## Null Deviance:      3311
## Residual Deviance: 3085 AIC: 3097
## log likelihood: -1542.451
## Nagelkerke R2: 0.09570436
## % pres/err predicted correctly: -910.0511
## % of predictable range [ (model-null)/(1-null) ]: 0.06099506
## *****
## model index: 7
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)      pos
##      1.28727      -0.14331      -0.05303      0.75311
##
## Degrees of Freedom: 4271 Total (i.e. Null); 4268 Residual
## Null Deviance:      3311
## Residual Deviance: 3090 AIC: 3098
## log likelihood: -1544.831
## Nagelkerke R2: 0.09374385
## % pres/err predicted correctly: -912.0972
## % of predictable range [ (model-null)/(1-null) ]: 0.05888619
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      pos
##      1.9583      -0.1588      0.3426
##
## Degrees of Freedom: 4271 Total (i.e. Null); 4269 Residual
## Null Deviance:      3311
## Residual Deviance: 3114 AIC: 3120
## log likelihood: -1557.151
## Nagelkerke R2: 0.08356172
## % pres/err predicted correctly: -919.9952
## % of predictable range [ (model-null)/(1-null) ]: 0.05074579
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)      pos

```

```

##      0.24058      -0.05991      0.76966
##
## Degrees of Freedom: 4271 Total (i.e. Null);  4269 Residual
## Null Deviance:      3311
## Residual Deviance: 3115  AIC: 3121
## log likelihood:  -1557.441
## Nagelkerke R2:  0.08332146
## % pres/err predicted correctly:  -920.1325
## % of predictable range [ (model-null)/(1-null) ]:  0.05060432
## *****
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos  stimlen:pos
##      1.65292      -0.12151      0.46611      -0.01469
##
## Degrees of Freedom: 4271 Total (i.e. Null);  4268 Residual
## Null Deviance:      3311
## Residual Deviance: 3114  AIC: 3122
## log likelihood:  -1556.785
## Nagelkerke R2:  0.083865
## % pres/err predicted correctly:  -920.0202
## % of predictable range [ (model-null)/(1-null) ]:  0.05072006
## *****
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      0.8728      0.3039
##
## Degrees of Freedom: 4271 Total (i.e. Null);  4270 Residual
## Null Deviance:      3311
## Residual Deviance: 3147  AIC: 3151
## log likelihood:  -1573.319
## Nagelkerke R2:  0.07010906
## % pres/err predicted correctly:  -929.8403
## % of predictable range [ (model-null)/(1-null) ]:  0.0405986
## *****
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)
##      1.896
##
## Degrees of Freedom: 4271 Total (i.e. Null);  4271 Residual

```



```
## Null Deviance:      3311
## Residual Deviance: 3311 AIC: 3313
## log likelihood: -1655.655
## Nagelkerke R2: 0
## % pres/err predicted correctly: -969.2303
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      2.10960      -0.02778
##
## Degrees of Freedom: 4271 Total (i.e. Null);  4270 Residual
## Null Deviance:      3311
## Residual Deviance: 3310 AIC: 3314
## log likelihood: -1655.117
## Nagelkerke R2: 0.0004673632
## % pres/err predicted correctly: -968.9911
## % of predictable range [ (model-null)/(1-null) ]: 0.0002466108
## *****
```

```
BestLPModel<-LPRes$ModelResult[[1]]
BestLPModelFormula<-LPRes$Model[[1]]

LPAICSummary<-data.frame(Model=LPRes$Model,
                          AIC=LPRes$AIC,
                          row.names = LPRes$Model)
LPAICSummary$DeltaAIC<-LPAICSummary$AIC-LPAICSummary$AIC[1]
LPAICSummary$AICexp<-exp(-0.5*LPAICSummary$DeltaAIC)
LPAICSummary$AICwt<-LPAICSummary$AICexp/sum(LPAICSummary$AICexp)
LPAICSummary$NagR2<-LPRes$NagR2

LPAICSummary <- merge(LPAICSummary,LPRes$CoefficientValues, by='row.names',sort=FALSE)
LPAICSummary <- subset(LPAICSummary, select = -c(Row.names))

write.csv(LPAICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                             "_len_pos_model_summary.csv"),row.names = FALSE)

kable(LPAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:I(pos)	I(pos^2)	stimlen:I(pos^2)
preserved ~ stimlen * (I(pos^2) + pos)	3096.902	0.0000000	0.0000000	0.5939172	0.2095702	2.0698244	-	0.4251170	0.0529561	-	-
							0.2571125			0.0554906	0.0015111
preserved ~ stimlen + I(pos^2) + pos	3097.603	0.7604208	0.6837105	0.4060706	0.0937433	2.872689	-	0.7531113	NA	-	NA
							0.1433075			0.0530256	
preserved ~ stimlen + pos	3120.302	23.3993303	0.0000000	0.0000000	0.0083561	7.9582938	-	0.3425757	NA	NA	NA
							0.1587643				
preserved ~ I(pos^2) + pos	3120.883	23.9790079	0.0000000	0.0000000	0.0708332	0.5240575	NA	0.7696604	NA	-	NA
										0.0599137	

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:I(pos^2)	I(pos^2)	stimlen:I(pos^2)
preserved ~ stimlen * pos	3121.5724	667498	0.000000	0.000000	0.000000	2.069824	-0.257112	-0.055491	0.425117	-0.001511	0.000000
preserved ~ pos	3150.6333	735952	0.000000	0.000000	0.000000	2.069824	-0.257112	-0.055491	0.425117	-0.001511	0.000000
preserved ~ 1	3313.3116	408263	0.000000	0.000000	0.000000	2.069824	-0.257112	-0.055491	0.425117	-0.001511	0.000000
preserved ~ stimlen	3314.2321	733126	0.000000	0.000000	0.000000	2.069824	-0.257112	-0.055491	0.425117	-0.001511	0.000000

```
print(BestLPModelFormula)
```

```
## [1] "preserved ~ stimlen * (I(pos^2) + pos)"
```

```
print(BestLPModel)
```

```
##
```

```
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
```

```
## data = PosDat)
```

```
##
```

```
## Coefficients:
```

```
## (Intercept)          stimlen          I(pos^2)          pos  stimlen:I(pos^2)          stimlen:I(pos^2)
```

```
## 2.069824          -0.257112          -0.055491          0.425117          -0.001511          0.000000
```

```
##
```

```
## Degrees of Freedom: 4271 Total (i.e. Null); 4266 Residual
```

```
## Null Deviance: 3311
```

```
## Residual Deviance: 3085 AIC: 3097
```

```
PosDat$LPFitted<-fitted(BestLPModel)
```

```
fitted_len_pos_plot <- plot_len_pos_obs_predicted(PosDat, PosDat$patient[1],  
NULL,palette_values=palette_values)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
# len/pos table
```

```
fitted_pos_len_summary <- PosDat %>% group_by(stimlen,pos) %>% summarise(fitted = mean(LPfitted))
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
fitted_pos_len_table <- fitted_pos_len_summary %>% pivot_wider(names_from = pos, values_from = fitted)  
fitted_pos_len_table
```

```
## # A tibble: 7 x 10
```

```
## # Groups:   stimlen [7]
```

```
## stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
```

```
## <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
```

```
## 1      4 0.834 0.888 0.917 NA      NA      NA      NA      NA      NA
```

```
## 2      5 0.804 0.871 0.908 0.927 NA      NA      NA      NA      NA
```

```
## 3      6 0.769 0.853 0.898 0.922 0.933 NA      NA      NA      NA
```

```
## 4      7 0.731 0.832 0.887 0.917 0.931 0.935 NA      NA      NA
```

```
## 5      8 0.689 0.808 0.876 0.911 0.929 0.935 0.934 NA      NA
```

```
## 6      9 0.643 0.783 0.863 0.905 0.927 0.936 0.936 0.927 NA
```

```
## 7     10 0.595 0.755 0.849 0.899 0.925 0.936 0.938 0.932 0.915
```

```
fitted_pos_len_summary$stimlen<-factor(fitted_pos_len_summary$stimlen)
```

```
fitted_pos_len_summary$pos<-factor(fitted_pos_len_summary$pos)
```

```
# fitted_len_pos_plot <- ggplot(pos_len_summary,aes(x=pos,y=preserved,group=stimlen,shape=stimlen,color=stimlen))
```

```

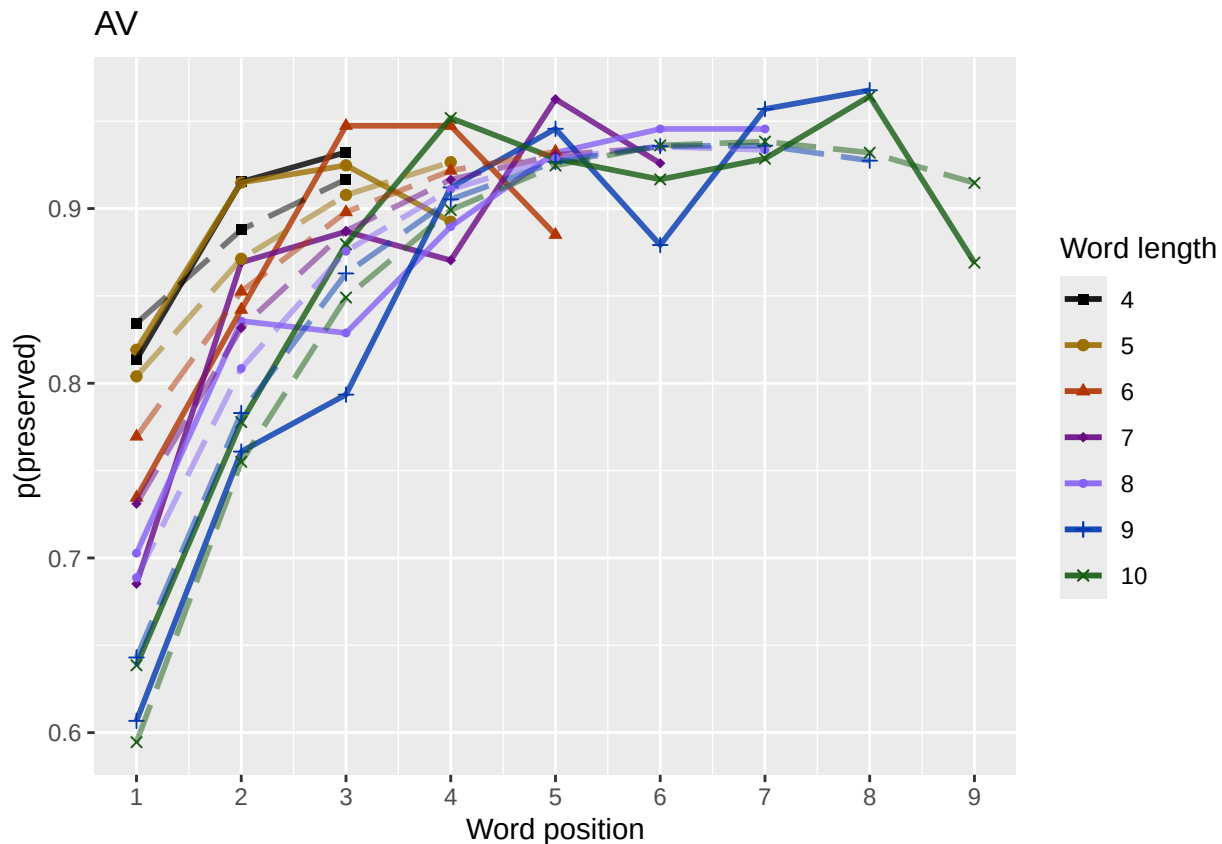
# fitted_len_pos_plot <- fitted_len_pos_plot + geom_line(data=fitted_pos_len_summary, aes(x=pos,y=fitted_pos_len_summary$y))
# geom_point(data=fitted_pos_len_summary,aes(x=pos,y=fitted_pos_len_summary$y,group=stimlen,shape=stimlen)) + ggtitle(paste0(
  patient, "LPFitted"))

fitted_len_pos_plot <- plot_len_pos_obs_predicted(PosDat,
  paste0(PosDat$patient[1],
    "LPFitted",
    NULL,
    palette_values,
    shape_values,
    obs_linetypes,
    pred_linetypes = c("longdash")
  )

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask, "_percent_preserved_by_length_pos_wfit.png"), plot=fitted_len_pos_plot

```



length and position without fragments to see if this changes position² influence

```

# first number responses, then count resp with fragments - below we will eliminate fragments
# and re-run models

# number responses
resp_num<-0
prev_pos<-9999 # big number to initialize (so first position is smaller)

```

```

resp_num_array <- integer(length = nrow(PosDat))
for(i in seq(1,nrow(PosDat))){
  if(PosDat$pos[i] <= prev_pos){ # equal for resp length 1
    resp_num <- resp_num + 1
  }
  resp_num_array[i]<-resp_num
  prev_pos <- PosDat$pos[i]
}
PosDat$resp_num <- resp_num_array

# count responses with fragments
resp_with_frag <- PosDat %>% group_by(resp_num) %>% summarise(frag = as.logical(sum(fragment)))
num_frag <- resp_with_frag %>% summarise(frag_sum = sum(frag), N=n())
num_frag

## # A tibble: 1 x 2
##   frag_sum      N
##   <int> <int>
## 1       5    700

num_frag$percent_with_frag[1] <- (num_frag$frag_sum[1] / num_frag$N[1])*100

## Warning: Unknown or uninitialised column: `percent_with_frag`.

print(sprintf("The number of responses with fragments was %d / %d = %.2f percent",
  num_frag$frag_sum[1],num_frag$N[1],num_frag$percent_with_frag[1]))

## [1] "The number of responses with fragments was 5 / 700 = 0.71 percent"

write.csv(num_frag,paste0(TablesDir,CurPat,"_",RunLabel,"_",
  CurTask,"_percent_fragments.csv"),row.names = FALSE)

# length and position models for data without fragments
NoFragData <- PosDat %>% filter(fragment != 1)

NoFrag_LPRes<-TestModels(LPModelEquations,NoFragData)

## *****
## model index: 8
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##   data = PosDat)
##
## Coefficients:
##   (Intercept)      stimlen      I(pos^2)      pos  stimlen:I(pos^2)      stimlen
##   2.049670      -0.255615      -0.053012      0.436408      -0.001531      0.
##
## Degrees of Freedom: 4260 Total (i.e. Null); 4255 Residual
## Null Deviance: 3266
## Residual Deviance: 3025 AIC: 3037
## log likelihood: -1512.597
## Nagelkerke R2: 0.1027692
## % pres/err predicted correctly: -890.7593
## % of predictable range [ (model-null)/(1-null) ]: 0.06480981
## *****
## model index: 7

```

```

##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)          pos
##      1.29611      -0.14631      -0.05138      0.75843
##
## Degrees of Freedom: 4260 Total (i.e. Null);  4257 Residual
## Null Deviance:      3266
## Residual Deviance: 3029  AIC: 3037
## log likelihood:  -1514.737
## Nagelkerke R2:  0.1009958
## % pres/err predicted correctly:  -892.6607
## % of predictable range [ (model-null)/(1-null) ]:  0.06281578
## *****
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##      1.9264      -0.1603      0.3654
##
## Degrees of Freedom: 4260 Total (i.e. Null);  4258 Residual
## Null Deviance:      3266
## Residual Deviance: 3051  AIC: 3057
## log likelihood:  -1525.557
## Nagelkerke R2:  0.09199968
## % pres/err predicted correctly:  -899.8089
## % of predictable range [ (model-null)/(1-null) ]:  0.05531938
## *****
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos  stimlen:pos
##      1.66328      -0.12815      0.47346      -0.01289
##
## Degrees of Freedom: 4260 Total (i.e. Null);  4257 Residual
## Null Deviance:      3266
## Residual Deviance: 3051  AIC: 3059
## log likelihood:  -1525.291
## Nagelkerke R2:  0.09222186
## % pres/err predicted correctly:  -899.8532
## % of predictable range [ (model-null)/(1-null) ]:  0.05527294
## *****
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)

```

```

##
## Coefficients:
## (Intercept)      I(pos^2)          pos
##      0.22729      -0.05839      0.77517
##
## Degrees of Freedom: 4260 Total (i.e. Null);  4258 Residual
## Null Deviance:      3266
## Residual Deviance: 3055  AIC: 3061
## log likelihood:  -1527.725
## Nagelkerke R2:  0.09019162
## % pres/err predicted correctly:  -900.8801
## % of predictable range [ (model-null)/(1-null) ]:  0.0541961
## *****
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      0.828      0.327
##
## Degrees of Freedom: 4260 Total (i.e. Null);  4259 Residual
## Null Deviance:      3266
## Residual Deviance: 3084  AIC: 3088
## log likelihood:  -1541.816
## Nagelkerke R2:  0.07839643
## % pres/err predicted correctly:  -909.7365
## % of predictable range [ (model-null)/(1-null) ]:  0.04490836
## *****
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)
##      1.915
##
## Degrees of Freedom: 4260 Total (i.e. Null);  4260 Residual
## Null Deviance:      3266
## Residual Deviance: 3266  AIC: 3268
## log likelihood:  -1633.17
## Nagelkerke R2:  0
## % pres/err predicted correctly:  -952.5593
## % of predictable range [ (model-null)/(1-null) ]:  0
## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen

```

```
##      2.1011      -0.0241
##
## Degrees of Freedom: 4260 Total (i.e. Null); 4259 Residual
## Null Deviance:      3266
## Residual Deviance: 3266 AIC: 3270
## log likelihood: -1632.771
## Nagelkerke R2: 0.000349593
## % pres/err predicted correctly: -952.385
## % of predictable range [ (model-null)/(1-null) ]: 0.0001826933
## *****
```

```
NoFragBestLPModel<-NoFrag_LPRes$ModelResult[[1]]
NoFragBestLPModelFormula<-NoFrag_LPRes$Model[[1]]

NoFragLPAICSummary<-data.frame(Model=NoFrag_LPRes$Model,
                                AIC=NoFrag_LPRes$AIC,
                                row.names = NoFrag_LPRes$Model)
NoFragLPAICSummary$DeltaAIC<-NoFragLPAICSummary$AIC-NoFragLPAICSummary$AIC[1]
NoFragLPAICSummary$AICexp<-exp(-0.5*NoFragLPAICSummary$DeltaAIC)
NoFragLPAICSummary$AICwt<-NoFragLPAICSummary$AICexp/sum(NoFragLPAICSummary$AICexp)
NoFragLPAICSummary$NagR2<-NoFrag_LPRes$NagR2

NoFragLPAICSummary <- merge(NoFragLPAICSummary,NoFrag_LPRes$CoefficientValues, by='row.names',sort=FALSE)
NoFragLPAICSummary <- subset(NoFragLPAICSummary, select = -c(Row.names))

write.csv(NoFragLPAICSummary,paste0(TablesDir,
                                    CurPat,"_",RunLabel,"_",
                                    CurTask,
                                    "_no_fragments_len_pos_model_summary.csv"),
          row.names = FALSE)
kable(NoFragLPAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:I(pos)	stimlen:I(pos^2)	stimlen:I(pos^2)
preserved ~ stimlen * (I(pos^2) + pos)	3037.195	0.000000	0.000000	0.000000	0.5347956	1027622	0.496699	- 0.4364078	0.0516071	-	-
							0.2556146			0.0530118	0.0015311
preserved ~ stimlen + I(pos^2) + pos	3037.473	0.278989	0.869797	0.146516	0.1100993	182961122	- 0.7584289	NA	-	NA	NA
							0.1463061			0.0513776	
preserved ~ stimlen + pos	3057.115	0.920176	0.000047	0.000025	0.3091999	179263626	- 0.3653572	NA	NA	NA	NA
							0.1603093				
preserved ~ stimlen * pos	3058.582	1.387002	0.000027	0.000012	0.1092221	196632799	- 0.4734650	-	NA	NA	NA
							0.1281487		0.0128897		
preserved ~ I(pos^2) + pos	3061.452	1.256400	0.000005	0.000002	0.0901906	2272941	NA	0.7751710	NA	-	NA
										0.0583917	
preserved ~ pos	3087.633	0.436870	0.000000	0.000000	0.0078396	48280480	NA	0.3270184	NA	NA	NA
preserved ~ 1	3268.342	1.145706	0.000000	0.000000	0.000000	0.09154159	NA	NA	NA	NA	NA
preserved ~ stimlen	3269.542	2.348098	0.000000	0.000000	0.0003426	1010727	-	NA	NA	NA	NA
							0.0241048				

```
# plot no fragment data
```

```
NoFragData$LPFitted<-fitted(NoFragBestLPModel)
```

```
nofrag_obs_len_pos_plot <- plot_len_pos_obs_predicted(NoFragData, NoFragData$patient[1],
  NULL,palette_values=palette_values)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
# len/pos table
```

```
nofrag_fitted_pos_len_summary <- NoFragData %>% group_by(stimlen,pos) %>% summarise(fitted = mean(LPFit
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
nofrag_fitted_pos_len_table <- nofrag_fitted_pos_len_summary %>% pivot_wider(names_from = pos, values_f
nofrag_fitted_pos_len_table
```

```
## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1     4 0.834 0.889 0.919 NA     NA     NA     NA     NA     NA
## 2     5 0.803 0.872 0.910 0.929 NA     NA     NA     NA     NA
## 3     6 0.768 0.853 0.900 0.924 0.937 NA     NA     NA     NA
## 4     7 0.730 0.832 0.889 0.919 0.934 0.940 NA     NA     NA
## 5     8 0.688 0.809 0.877 0.914 0.932 0.940 0.940 NA     NA
## 6     9 0.642 0.783 0.864 0.908 0.930 0.940 0.942 0.936 NA
## 7    10 0.593 0.755 0.850 0.901 0.927 0.940 0.943 0.939 0.926
```

```
fitted_pos_len_summary$stimlen<-factor(nofrag_fitted_pos_len_summary$stimlen)
```

```
fitted_pos_len_summary$pos<-factor(nofrag_fitted_pos_len_summary$pos)
```

```
# fitted_len_pos_plot <- ggplot(pos_len_summary,aes(x=pos,y=preserved,group=stimlen,shape=stimlen,color=
```

```
# fitted_len_pos_plot <- fitted_len_pos_plot + geom_line(data=fitted_pos_len_summary, aes(x=pos,y=fitted
```

```
# geom_point(data=fitted_pos_len_summary,aes(x=pos,y=fitted,group=stimlen,shape=stimlen)) + ggtitle(pas
```

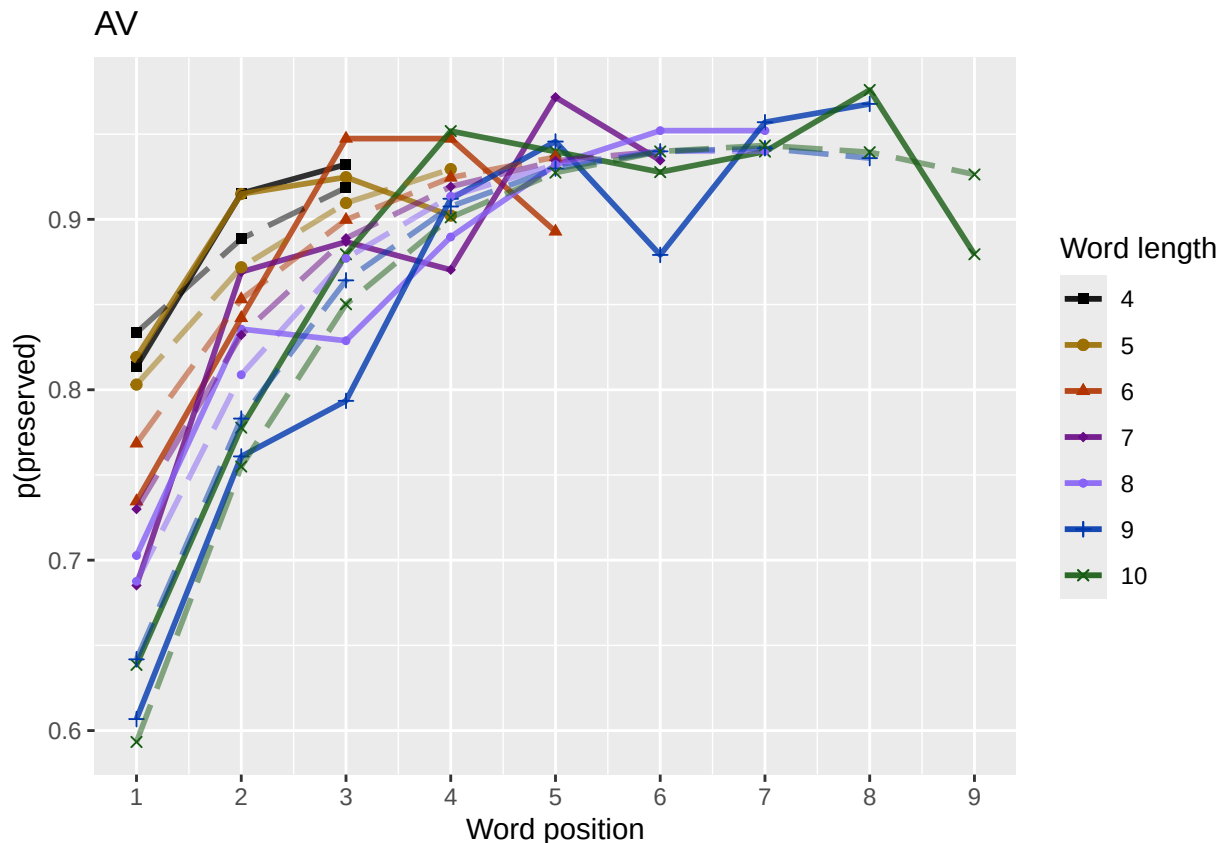
```
nofrag_fitted_len_pos_plot <- plot_len_pos_obs_predicted(NoFragData,
  paste0(NoFragData$patient[1]),
  "LPFitted",
  NULL,
  palette_values,
  shape_values,
  obs_linetypes,
  pred_linetypes = c("longdash")
)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,
  "no_fragments_percent_preserved_by_length_pos_wfit.png"),
  plot=nofrag_fitted_len_pos_plot,
  device="png",unit="cm",
  width=15,height=11)
nofrag_fitted_len_pos_plot
```

back to full data

```
results_report_DF <- AddReportLine(results_report_DF,"min preserved",min_preserved)
results_report_DF <- AddReportLine(results_report_DF,"max preserved",max_preserved)
print(sprintf("Min/max preserved range: %.2f - %.2f",min_preserved,max_preserved))
```

```
## [1] "Min/max preserved range: 0.57 - 1.00"
```

```
# find the average difference between lengths and the average difference
# between positions taking the other factor into account
```

```
# choose fitted or raw -- we use fitted values because they are smoothed averages
# that take into account the influence of the other variable
```

```
table_to_use <- fitted_pos_len_table
# take away column with length information
table_to_use <- table_to_use[,2:ncol(table_to_use)]
```

```
# take averages for positions
```

```
# don't want downward estimates influenced by return upward of U
```

```
# therefore, for downward influence, use only the values before the min
```

```
# take the difference between each value (differences between position proportion correct) **NOTE** pro
```

```
# average the difference in probabilities
```

```
# in case min is close to the end or we are not using a min (for non-U-shaped)
```

```
# functions, we need to get differences _first_ and then average those (e.g.
```

```
# if there is an effect of length and we just average position data,
```

```
# later positions) will have a lower average (because of the length effect)
```

```
# even if all position functions go upward
```

```

table_pos_diffs <- t(diff(t(as.matrix(table_to_use))))
ncol_pos_diff_matrix <- ncol(table_pos_diffs)
first_col_mean <- mean(as.numeric(unlist(table_pos_diffs[,1])))
potential_u_shape <- 0
if(first_col_mean < 0){ # downward, so potential U-shape
  # take only negative values from the table
  filtered_table_pos_diffs<-table_pos_diffs
  filtered_table_pos_diffs[table_pos_diffs >= 0] <- NA
  potential_u_shape <- 1
}else{
  # upward - use the positive values
  # take only negative values from the table
  filtered_table_pos_diffs<-table_pos_diffs
  filtered_table_pos_diffs[table_pos_diffs <= 0] <- NA
}
average_pos_diffs <- apply(filtered_table_pos_diffs,2,mean,na.rm=TRUE)
OA_mean_pos_diff <- mean(average_pos_diffs,na.rm=TRUE)

table_len_diffs <- diff(as.matrix(table_to_use))
average_len_diffs <- apply(table_len_diffs,1,mean,na.rm=TRUE)
OA_mean_len_diff <- mean(average_len_diffs,na.rm=TRUE)
CurrentLabel<-"mean change in probability for each additional length: "
print(CurrentLabel)

## [1] "mean change in probability for each additional length: "
print(OA_mean_len_diff)

## [1] -0.01486361
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,OA_mean_len_diff)

CurrentLabel<-"mean change in probability for each additional position (excluding U-shape): "
print(CurrentLabel)

## [1] "mean change in probability for each additional position (excluding U-shape): "
print(OA_mean_pos_diff)

## [1] 0.03700757
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,OA_mean_pos_diff)

if(potential_u_shape){ # downward, so potential U-shape
  # take only positive values from the table
  filtered_pos_upward_u<-table_pos_diffs
  filtered_pos_upward_u[table_pos_diffs <= 0] <- NA
  average_pos_u_diffs <- apply(filtered_pos_upward_u,
                              2,mean,na.rm=TRUE)
  OA_mean_pos_u_diff <- mean(average_pos_u_diffs,na.rm=TRUE)
  if(is.nan(OA_mean_pos_u_diff) | (OA_mean_pos_u_diff <= 0)){
    print("No U-shape in this participant")
    results_report_DF <- AddReportLine(results_report_DF, "U-shape", 0)
    potential_u_shape <- FALSE
  }else{
    results_report_DF <- AddReportLine(results_report_DF, "U-shape", 1)
  }
}

```

```

CurrentLabel<-"Average upward change after U minimum"
print(CurrentLabel)
print(OA_mean_pos_u_diff)
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, OA_mean_pos_u_diff)

CurrentLabel<-"Proportion of average downward change"
prop_ave_down <- abs(OA_mean_pos_u_diff/OA_mean_pos_diff)
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, prop_ave_down)
}
}else{
  print("No U-shape in this participant")
  results_report_DF <- AddReportLine(results_report_DF, "U-shape", 0)
}

## [1] "No U-shape in this participant"
if(potential_u_shape){
  n_rows<-nrow(table_to_use)
  downward_dist <- numeric(n_rows)
  upward_dist <- numeric(n_rows)
  for(i in seq(1,n_rows)){
    current_row <- as.numeric(unlist(table_to_use[i,1:9]))
    current_row <- current_row[!is.na(current_row)]
    current_row_len <- length(current_row)
    row_min <- min(current_row,na.rm=TRUE)
    min_pos <- which(current_row == row_min)
    left_max <- max(current_row[1:min_pos],na.rm=TRUE)
    right_max <- max(current_row[min_pos:current_row_len])
    left_diff <- left_max - row_min
    right_diff <- right_max - row_min
    downward_dist[i]<-left_diff
    upward_dist[i]<-right_diff
  }
  print("differences from left max to min for each row: ")
  print(downward_dist)
  print("differences from min to right max for each row: ")
  print(upward_dist)

  # Use the max return upward (min of upward_dist) and then the
  # downward distance from the left for the same row

  print(" ")
  biggest_return_upward_row <- which(upward_dist == max(upward_dist))
  CurrentLabel <- "row with biggest return upward"
  print(CurrentLabel)
  print(biggest_return_upward_row)
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, biggest_return_upward_row)

  print(" ")
  CurrentLabel<-"downward distance for row with the largest upward value"
  print(CurrentLabel)
  print(downward_dist[biggest_return_upward_row])
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, downward_dist[biggest_return_upward_row])
}

```

```

CurrentLabel<-"return upward value"
print(upward_dist[biggest_return_upward_row])
results_report_DF <- AddReportLine(results_report_DF,
                                   CurrentLabel,
                                   upward_dist[biggest_return_upward_row])

print(" ")
# percentage return upward
percentage_return_upward <- upward_dist[biggest_return_upward_row]/downward_dist[biggest_return_upward_row]
CurrentLabel <- "Return upward as a proportion of the downward distance:"
print(CurrentLabel)
print(percentange_return_upward)
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,
                                   percentange_return_upward)
}else{
  print("no U-shape in this participant")
}

## [1] "no U-shape in this participant"

FLPModelEquations<-c(
  # models with log frequency, but no three-way interactions (due to difficulty interpreting and small sample sizes)
  "preserved ~ stimlen*log_freq",
  "preserved ~ stimlen+log_freq",
  "preserved ~ pos*log_freq",
  "preserved ~ pos+log_freq",
  "preserved ~ stimlen*log_freq + pos*log_freq",
  "preserved ~ stimlen*log_freq + pos",
  "preserved ~ stimlen + pos*log_freq",
  "preserved ~ stimlen + pos + log_freq",
  "preserved ~ (I(pos^2)+pos)*log_freq",
  "preserved ~ stimlen*log_freq + (I(pos^2) + pos)*log_freq",
  "preserved ~ stimlen*log_freq + I(pos^2) + pos",
  "preserved ~ stimlen + (I(pos^2) + pos)*log_freq",
  "preserved ~ stimlen + I(pos^2) + pos + log_freq",

  # models without frequency
  "preserved ~ 1",
  "preserved ~ stimlen",
  "preserved ~ pos",
  "preserved ~ stimlen + pos",
  "preserved ~ stimlen*pos",
  "preserved ~ I(pos^2)+pos",
  "preserved ~ stimlen + I(pos^2) + pos",
  "preserved ~ stimlen * (I(pos^2) + pos)"
)

FLPRes<-TestModels(FLPModelEquations,PosDat)

## *****
## model index: 11
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##

```

```

## Coefficients:
##      (Intercept)      stimlen      log_freq      I(pos^2)      pos      stimlen:log
##      0.96449      -0.10991      0.58758      -0.05530      0.77203      -0
##
## Degrees of Freedom: 4271 Total (i.e. Null);  4266 Residual
## Null Deviance:      3311
## Residual Deviance: 3059  AIC: 3071
## log likelihood: -1529.418
## Nagelkerke R2:  0.1064005
## % pres/err predicted correctly: -905.1984
## % of predictable range [ (model-null)/(1-null) ]:  0.06599663
## *****
## model index:  10
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      log_freq      I(pos^2)      pos      stim
##      0.969226      -0.112514      0.538916      -0.055696      0.779677
##      log_freq:pos
##      0.042205
##
## Degrees of Freedom: 4271 Total (i.e. Null);  4264 Residual
## Null Deviance:      3311
## Residual Deviance: 3057  AIC: 3073
## log likelihood: -1528.636
## Nagelkerke R2:  0.1070404
## % pres/err predicted correctly: -905.2247
## % of predictable range [ (model-null)/(1-null) ]:  0.06596954
## *****
## model index:  13
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      I(pos^2)      pos      log_freq
##      1.06553      -0.11439      -0.05351      0.75782      0.08838
##
## Degrees of Freedom: 4271 Total (i.e. Null);  4267 Residual
## Null Deviance:      3311
## Residual Deviance: 3078  AIC: 3088
## log likelihood: -1538.852
## Nagelkerke R2:  0.0986647
## % pres/err predicted correctly: -909.7139
## % of predictable range [ (model-null)/(1-null) ]:  0.06134255
## *****
## model index:  12
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:

```

```

##      (Intercept)          stimlen          I(pos^2)          pos          log_freq  I(pos
##      1.034647          -0.112455          -0.056098          0.774060          0.035482
##
## Degrees of Freedom: 4271 Total (i.e. Null); 4265 Residual
## Null Deviance:          3311
## Residual Deviance: 3076 AIC: 3090
## log likelihood: -1538.179
## Nagelkerke R2: 0.09921788
## % pres/err predicted correctly: -909.6548
## % of predictable range [ (model-null)/(1-null) ]: 0.06140354
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
##      (Intercept)          stimlen          log_freq          pos  stimlen:log_freq          log_fr
##      1.67293          -0.12991          0.55180          0.35249          -0.06790          0
##
## Degrees of Freedom: 4271 Total (i.e. Null); 4266 Residual
## Null Deviance:          3311
## Residual Deviance: 3083 AIC: 3095
## log likelihood: -1541.497
## Nagelkerke R2: 0.09648934
## % pres/err predicted correctly: -913.2658
## % of predictable range [ (model-null)/(1-null) ]: 0.05768168
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
##      (Intercept)          stimlen          log_freq          pos  stimlen:log_freq
##      1.67496          -0.12739          0.56902          0.34442          -0.06222
##
## Degrees of Freedom: 4271 Total (i.e. Null); 4267 Residual
## Null Deviance:          3311
## Residual Deviance: 3085 AIC: 3095
## log likelihood: -1542.571
## Nagelkerke R2: 0.09560576
## % pres/err predicted correctly: -913.497
## % of predictable range [ (model-null)/(1-null) ]: 0.05744344
## *****
## model index: 21
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
##      (Intercept)          stimlen          I(pos^2)          pos  stimlen:I(pos^2)          stiml
##      2.069824          -0.257112          -0.055491          0.425117          -0.001511          0.
##

```

```

## Degrees of Freedom: 4271 Total (i.e. Null); 4266 Residual
## Null Deviance: 3311
## Residual Deviance: 3085 AIC: 3097
## log likelihood: -1542.451
## Nagelkerke R2: 0.09570436
## % pres/err predicted correctly: -910.0511
## % of predictable range [ (model-null)/(1-null) ]: 0.06099506
## *****
## model index: 20
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen I(pos^2) pos
## 1.28727 -0.14331 -0.05303 0.75311
##
## Degrees of Freedom: 4271 Total (i.e. Null); 4268 Residual
## Null Deviance: 3311
## Residual Deviance: 3090 AIC: 3098
## log likelihood: -1544.831
## Nagelkerke R2: 0.09374385
## % pres/err predicted correctly: -912.0972
## % of predictable range [ (model-null)/(1-null) ]: 0.05888619
## *****
## model index: 9
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) I(pos^2) pos log_freq I(pos^2):log_freq
## 0.206780 -0.061781 0.790373 0.068818 -0.006588
##
## Degrees of Freedom: 4271 Total (i.e. Null); 4266 Residual
## Null Deviance: 3311
## Residual Deviance: 3090 AIC: 3102
## log likelihood: -1545.163
## Nagelkerke R2: 0.0934706
## % pres/err predicted correctly: -914.0168
## % of predictable range [ (model-null)/(1-null) ]: 0.05690762
## *****
## model index: 8
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen pos log_freq
## 1.74576 -0.13042 0.34356 0.08676
##
## Degrees of Freedom: 4271 Total (i.e. Null); 4268 Residual
## Null Deviance: 3311
## Residual Deviance: 3103 AIC: 3111

```

```

## log likelihood: -1551.327
## Nagelkerke R2: 0.08838231
## % pres/err predicted correctly: -917.6498
## % of predictable range [ (model-null)/(1-null) ]: 0.05316317
## *****
## model index: 7
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen pos log_freq pos:log_freq
## 1.746290 -0.131084 0.345666 0.073085 0.004619
##
## Degrees of Freedom: 4271 Total (i.e. Null); 4267 Residual
## Null Deviance: 3311
## Residual Deviance: 3103 AIC: 3113
## log likelihood: -1551.268
## Nagelkerke R2: 0.08843165
## % pres/err predicted correctly: -917.6905
## % of predictable range [ (model-null)/(1-null) ]: 0.0531212
## *****
## model index: 17
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen pos
## 1.9583 -0.1588 0.3426
##
## Degrees of Freedom: 4271 Total (i.e. Null); 4269 Residual
## Null Deviance: 3311
## Residual Deviance: 3114 AIC: 3120
## log likelihood: -1557.151
## Nagelkerke R2: 0.08356172
## % pres/err predicted correctly: -919.9952
## % of predictable range [ (model-null)/(1-null) ]: 0.05074579
## *****
## model index: 19
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) I(pos^2) pos
## 0.24058 -0.05991 0.76966
##
## Degrees of Freedom: 4271 Total (i.e. Null); 4269 Residual
## Null Deviance: 3311
## Residual Deviance: 3115 AIC: 3121
## log likelihood: -1557.441
## Nagelkerke R2: 0.08332146
## % pres/err predicted correctly: -920.1325

```



```

## % of predictable range [ (model-null)/(1-null) ]: 0.05060432
## *****
## model index: 18
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos  stimlen:pos
##      1.65292      -0.12151      0.46611      -0.01469
##
## Degrees of Freedom: 4271 Total (i.e. Null); 4268 Residual
## Null Deviance:      3311
## Residual Deviance: 3114 AIC: 3122
## log likelihood: -1556.785
## Nagelkerke R2: 0.083865
## % pres/err predicted correctly: -920.0202
## % of predictable range [ (model-null)/(1-null) ]: 0.05072006
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          pos      log_freq
##      0.8445      0.3156      0.1193
##
## Degrees of Freedom: 4271 Total (i.e. Null); 4269 Residual
## Null Deviance:      3311
## Residual Deviance: 3122 AIC: 3128
## log likelihood: -1561.219
## Nagelkerke R2: 0.08018628
## % pres/err predicted correctly: -923.5136
## % of predictable range [ (model-null)/(1-null) ]: 0.0471195
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          pos      log_freq  pos:log_freq
##      0.8437347      0.3159935      0.1170042      0.0008023
##
## Degrees of Freedom: 4271 Total (i.e. Null); 4268 Residual
## Null Deviance:      3311
## Residual Deviance: 3122 AIC: 3130
## log likelihood: -1561.217
## Nagelkerke R2: 0.08018775
## % pres/err predicted correctly: -923.5268
## % of predictable range [ (model-null)/(1-null) ]: 0.04710583
## *****
## model index: 16

```

```

##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      0.8728      0.3039
##
## Degrees of Freedom: 4271 Total (i.e. Null);  4270 Residual
## Null Deviance:      3311
## Residual Deviance: 3147  AIC: 3151
## log likelihood:  -1573.319
## Nagelkerke R2:  0.07010906
## % pres/err predicted correctly:  -929.8403
## % of predictable range [ (model-null)/(1-null) ]:  0.0405986
## *****
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      log_freq  stimlen:log_freq
##      1.839784      0.002803      0.550247      -0.060035
##
## Degrees of Freedom: 4271 Total (i.e. Null);  4268 Residual
## Null Deviance:      3311
## Residual Deviance: 3282  AIC: 3290
## log likelihood:  -1641.06
## Nagelkerke R2:  0.01262608
## % pres/err predicted correctly:  -963.0705
## % of predictable range [ (model-null)/(1-null) ]:  0.006348857
## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      log_freq
##  1.9071918  -0.0003753   0.0828603
##
## Degrees of Freedom: 4271 Total (i.e. Null);  4269 Residual
## Null Deviance:      3311
## Residual Deviance: 3299  AIC: 3305
## log likelihood:  -1649.533
## Nagelkerke R2:  0.005306966
## % pres/err predicted correctly:  -966.5802
## % of predictable range [ (model-null)/(1-null) ]:  0.002731434
## *****
## model index:  14
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)

```

```
##
## Coefficients:
## (Intercept)
##      1.896
##
## Degrees of Freedom: 4271 Total (i.e. Null);  4271 Residual
## Null Deviance:      3311
## Residual Deviance: 3311 AIC: 3313
## log likelihood:  -1655.655
## Nagelkerke R2:  0
## % pres/err predicted correctly:  -969.2303
## % of predictable range [ (model-null)/(1-null) ]:  0
## *****
## model index:  15
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      2.10960      -0.02778
##
## Degrees of Freedom: 4271 Total (i.e. Null);  4270 Residual
## Null Deviance:      3311
## Residual Deviance: 3310 AIC: 3314
## log likelihood:  -1655.117
## Nagelkerke R2:  0.0004673632
## % pres/err predicted correctly:  -968.9911
## % of predictable range [ (model-null)/(1-null) ]:  0.0002466108
## *****

BestFLPModel<-FLPRes$ModelResult[[1]]
BestFLPModelFormula<-FLPRes$Model[[1]]

FLPAICSummary<-data.frame(Model=FLPRes$Model,
                          AIC=FLPRes$AIC,row.names=FLPRes$Model)
FLPAICSummary$DeltaAIC<-FLPAICSummary$AIC-FLPAICSummary$AIC[1]
FLPAICSummary$AICexp<-exp(-0.5*FLPAICSummary$DeltaAIC)
FLPAICSummary$AICwt<-FLPAICSummary$AICexp/sum(FLPAICSummary$AICexp)
FLPAICSummary$NagR2<-FLPRes$NagR2

FLPAICSummary <- merge(FLPAICSummary,FLPRes$CoefficientValues,
                      by='row.names',sort=FALSE)
FLPAICSummary <- subset(FLPAICSummary, select = -c(Row.names))

write.csv(FLPAICSummary,paste0(TablesDir,CurPat,"_",
                              RunLabel,"_",CurTask,
                              "_freq_len_pos_model_summary.csv"),row.names = FALSE)

kable(FLPAICSummary)
```

Model	AIC Delta	AIC	AICw	NagR ²	Intercept	log_stimlen	log_pos	log_freq	I(pos^2)	log_freq	I(pos^2)	log_freq	I(pos^2)	len:I(pos^2)
preserved ~ stimlen * log_freq + I(pos^2) + pos	3070.836	0.000000	0.000000	0.516619	0.544929	0.5875788	0.7728309	NA	-	NA	NA	NA	NA	NA
						0.1099135	0.0642684			0.0552989				
preserved ~ stimlen * log_freq + (I(pos^2) + pos) * log_freq	3073.272	2.557795	0.228274	0.4692256	0.5389164	0.7798769	0.0422046	NA	-	NA	NA	NA	NA	NA
						0.1125135	0.0683498			0.0556963	0.0035243			
preserved ~ stimlen + I(pos^2) + pos + log_freq	3087.764	16.963702	1.000678	0.6655297	0.0883774	0.7578249	NA	-	NA	NA	NA	NA	NA	NA
						0.1143895				0.0535130				
preserved ~ stimlen + (I(pos^2) + pos) * log_freq	3090.158	20.228003	1.004992	0.6470	0.0354821	0.7740603	NA	-	-	NA	NA	NA	NA	NA
						0.1124552				0.0560980	0.059643			
preserved ~ stimlen * log_freq + pos * log_freq	3094.291	15.830100	0.570096	0.48729312	0.5517954	0.3528878	0.0206632	NA	NA	NA	NA	NA	NA	NA
						0.1299099	0.0678973							
preserved ~ stimlen * log_freq + pos	3095.242	20.540100	0.530095	0.607849614	0.5690196	0.3444159	NA	NA	NA	NA	NA	NA	NA	NA
						0.1273886	0.0622167							
preserved ~ stimlen * (I(pos^2) + pos)	3096.262	16.592700	0.220095	0.70698244	NA	0.4251770	NA	-	NA	NA	0.0529561			0.0015111
						0.2571125				0.0554906				
preserved ~ stimlen + I(pos^2) + pos	3097.263	16.348000	0.150092	0.72872689	NA	0.7531713	NA	-	NA	NA	NA	NA	NA	NA
						0.1433075				0.0530256				
preserved ~ (I(pos^2) + pos) * log_freq	3102.324	18.943900	0.000934	0.7067797	0.0688177	0.7903734	NA	-	-	NA	NA	NA	NA	NA
										0.0610810	0.065882			
preserved ~ stimlen + pos + log_freq	3110.355	18.030000	0.000883	0.757578	0.0867599	0.3435573	NA	NA	NA	NA	NA	NA	NA	NA
						0.1304182								
preserved ~ stimlen + pos * log_freq	3112.435	19.863900	0.000881	0.7462990	0.0730847	0.3456658	NA	NA	NA	NA	NA	NA	NA	NA
						0.1310839								
preserved ~ stimlen + pos	3120.402	16.526900	0.000835	0.652938	NA	0.3425757	NA	NA	NA	NA	NA	NA	NA	NA
						0.1587643								

Model	AIC Delta	AIC	AICw	NagR ²	(Intercept)	log_stimlen	log_pos	log_freq	I(pos^2)	pos	log_freq:I(pos)	len:I(pos^2)
preserved ~ I(pos^2) + pos	3120.58	10493	0.0000000000000000	0.833245	0.972	NA	NA	0.769864	NA	-	NA	NA
										0.0599137		
preserved ~ stimlen * pos	3121.57	7384	0.0000000000000000	0.838652	0.9164	NA	NA	0.466117	NA	NA	NA	-
										0.1215132		0.0146867
preserved ~ pos + log_freq	3128.47	8009	0.0000000000000000	0.800834	0.99	0.119314	0.315835	NA	NA	NA	NA	NA
preserved ~ pos * log_freq	3130.43	5984	0.0000000000000000	0.800873	0.947	0.117004	0.315993	0.8023	NA	NA	NA	NA
preserved ~ pos	3150.73	8008	0.0000000000000000	0.700187	0.8195	NA	NA	0.303886	NA	NA	NA	NA
preserved ~ stimlen * log_freq	3290.21	9280	0.0000000000000000	0.262397	0.837	0.280530	0.2467	NA	NA	NA	NA	NA
										0.0600353		
preserved ~ stimlen + log_freq	3305.26	4208	0.0000000000000000	0.53907	0.1918	0.082864	0.3	NA	NA	NA	NA	NA
										0.0003753		
preserved ~ 1	3313.24	1474	0.0000000000000000	0.089550	0.8	NA	NA	NA	NA	NA	NA	NA
preserved ~ stimlen	3314.23	1397	0.0000000000000000	0.267096	0.646	NA	NA	NA	NA	NA	NA	NA
										0.0277784		

```
print(BestFLPModelFormula)
```

```
## [1] "preserved ~ stimlen * log_freq + I(pos^2) + pos"
```

```
print(BestFLPModel)
```

```
##
```

```
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",  
## data = PosDat)
```

```
##
```

```
## Coefficients:
```

```
## (Intercept)          stimlen          log_freq          I(pos^2)          pos  stimlen:log  
##      0.96449         -0.10991         0.58758        -0.05530         0.77203         -0
```

```
##
```

```
## Degrees of Freedom: 4271 Total (i.e. Null); 4266 Residual
```

```
## Null Deviance: 3311
```

```
## Residual Deviance: 3059 AIC: 3071
```

```
# do a median split on frequency to plot hf/ls effects (analysis is continuous)
```

```
median_freq <- median(PosDat$log_freq)
```

```
PosDat$freq_bin[PosDat$log_freq >= median_freq]<-"hf"
```

```
PosDat$freq_bin[PosDat$log_freq < median_freq]<-"lf"
```

```
PosDat$FLPFitted<-fitted(BestFLPModel)
```

```
HFDat <- PosDat[PosDat$freq_bin == "hf",]
```

```
LFDat <- PosDat[PosDat$freq_bin == "lf",]
```

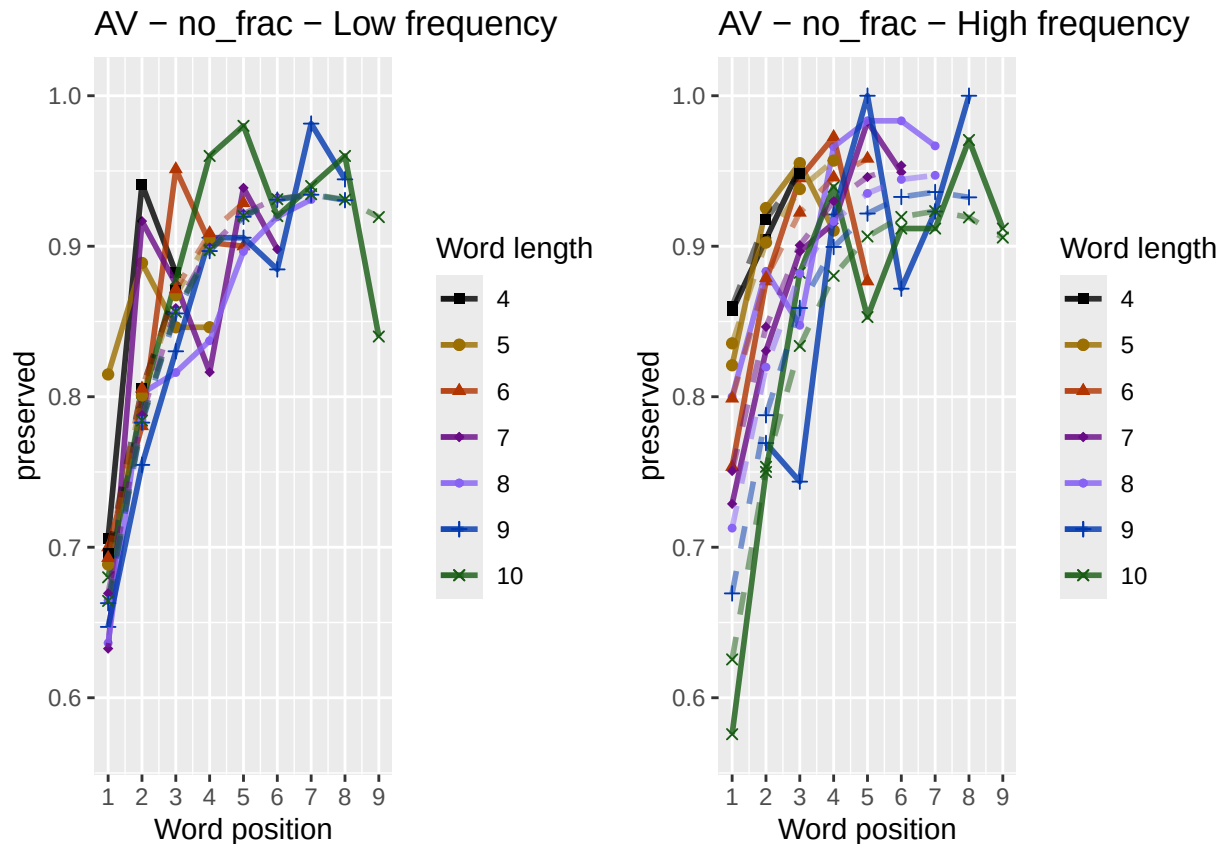
```
HF_Plot <- plot_len_pos_obs_predicted(HFDat,paste0(CurPat," - ",RunLabel," - High frequency"),"FLPFitted"
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
LF_Plot <- plot_len_pos_obs_predicted(LFdat, paste0(CurPat, " - ", RunLabel, " - Low frequency"), "FLPFitted")

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
library(ggpubr)
Both_Plots <- ggarrange(LF_Plot, HF_Plot) # labels=c("LF", "HF", ncol=2)

## Warning: Removed 1 row containing missing values or values outside the scale range (`geom_point()`).
## Warning: Removed 1 row containing missing values or values outside the scale range (`geom_line()`).
ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_",
              CurTask, "_frequency_effect_length_pos_wfit.png"),
       device="png", unit="cm", width=30, height=11)
print(Both_Plots)
```



```
# only main effects
MEModelEquations<-c(
  "preserved ~ CumPres",
  "preserved ~ CumErr",
  "preserved ~ (I(pos^2)+pos)",
  "preserved ~ pos",
  "preserved ~ stimlen",
```

```

"preserved ~ 1"
)
MERes<-TestModels(MEModelEquations,PosDat)

## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      1.1118      0.4498
##
## Degrees of Freedom: 4271 Total (i.e. Null); 4270 Residual
## Null Deviance:      3311
## Residual Deviance: 3055 AIC: 3059
## log likelihood: -1527.326
## Nagelkerke R2: 0.1081112
## % pres/err predicted correctly: -909.2099
## % of predictable range [ (model-null)/(1-null) ]: 0.06186207
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)      pos
##      0.24058      -0.05991      0.76966
##
## Degrees of Freedom: 4271 Total (i.e. Null); 4269 Residual
## Null Deviance:      3311
## Residual Deviance: 3115 AIC: 3121
## log likelihood: -1557.441
## Nagelkerke R2: 0.08332146
## % pres/err predicted correctly: -920.1325
## % of predictable range [ (model-null)/(1-null) ]: 0.05060432
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      pos
##      0.8728      0.3039
##
## Degrees of Freedom: 4271 Total (i.e. Null); 4270 Residual
## Null Deviance:      3311
## Residual Deviance: 3147 AIC: 3151
## log likelihood: -1573.319
## Nagelkerke R2: 0.07010906
## % pres/err predicted correctly: -929.8403

```

```

## % of predictable range [ (model-null)/(1-null) ]: 0.0405986
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumErr
## 1.9810 -0.1569
##
## Degrees of Freedom: 4271 Total (i.e. Null); 4270 Residual
## Null Deviance: 3311
## Residual Deviance: 3303 AIC: 3307
## log likelihood: -1651.342
## Nagelkerke R2: 0.003740309
## % pres/err predicted correctly: -967.1482
## % of predictable range [ (model-null)/(1-null) ]: 0.002146027
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
## 1.896
##
## Degrees of Freedom: 4271 Total (i.e. Null); 4271 Residual
## Null Deviance: 3311
## Residual Deviance: 3311 AIC: 3313
## log likelihood: -1655.655
## Nagelkerke R2: 0
## % pres/err predicted correctly: -969.2303
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen
## 2.10960 -0.02778
##
## Degrees of Freedom: 4271 Total (i.e. Null); 4270 Residual
## Null Deviance: 3311
## Residual Deviance: 3310 AIC: 3314
## log likelihood: -1655.117
## Nagelkerke R2: 0.0004673632
## % pres/err predicted correctly: -968.9911
## % of predictable range [ (model-null)/(1-null) ]: 0.0002466108
## *****

```



```

BestMEModel<-MERes$ModelResult[[ BestModelIndexL1 ]]
BestMEModelFormula<-MERes$Model[[BestModelIndexL1]]

MEAICSummary<-data.frame(Model=MERes$Model,
                          AIC=MERes$AIC,row.names=MERes$Model)
MEAICSummary$DeltaAIC<-MEAICSummary$AIC-MEAICSummary$AIC[1]
MEAICSummary$AICexp<-exp(-0.5*MEAICSummary$DeltaAIC)
MEAICSummary$AICwt<-MEAICSummary$AICexp/sum(MEAICSummary$AICexp)
MEAICSummary$NagR2<-MERes$NagR2

MEAICSummary <- merge(MEAICSummary,MERes$CoefficientValues,
                      by='row.names',sort=FALSE)
MEAICSummary <- subset(MEAICSummary, select = -c(Row.names))

write.csv(MEAICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                             CurTask,"_main_effects_model_summary.csv"),
          row.names = FALSE)
kable(MEAICSummary)

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumPres	3058.653	0.00000	1	1	0.108111	2.111770	0.4497816	NA	NA	NA	NA
preserved ~ (I(pos^2) + pos)	3120.8816	2.22847	0	0	0.0833216	0.2405752	NA	NA	- 0.0599137	0.7696604	NA
preserved ~ pos	3150.6339	1.98541	0	0	0.0701090	0.8728195	NA	NA	NA	0.3038868	NA
preserved ~ CumErr	3306.6842	48.03093	0	0	0.0037403	0.9809578	NA	- 0.156855	NA	NA	NA
preserved ~ 1	3313.3112	54.65771	0	0	0.0000000	0.8955058	NA	NA	NA	NA	NA
preserved ~ stimlen	3314.2342	55.58072	0	0	0.0004672	1.1096046	NA	NA	NA	NA	- 0.0277784

```

if(DoSimulations){
  BestMEModelFormulaRnd <- BestMEModelFormula
  if(grepl("CumPres",BestMEModelFormulaRnd)){
    BestMEModelFormulaRnd <- gsub("CumPres","RndCumPres",BestMEModelFormulaRnd)
  }else if(grepl("CumErr",BestMEModelFormulaRnd)){
    BestMEModelFormulaRnd <- gsub("CumErr","RndCumErr",BestMEModelFormulaRnd)
  }

  RndModelAIC<-numeric(length=RandomSamples)
  for(rindex in seq(1,RandomSamples)){
    # Shuffle cumulative values
    PosDat$RndCumPres <- ShuffleWithinWord(PosDat,"CumPres")
    PosDat$RndCumErr <- ShuffleWithinWord(PosDat,"CumErr")
    BestModelRnd <- glm(as.formula(BestMEModelFormulaRnd),
                       family="binomial",data=PosDat)
    RndModelAIC[rindex] <- BestModelRnd$aic
  }
  ModelNames<-c(paste0("***",BestMEModelFormula),
                rep(BestMEModelFormulaRnd,RandomSamples))
  AICValues <- c(BestMEModel$aic,RndModelAIC)
  BestMEModelRndDF <- data.frame(Name=ModelNames,AIC=AICValues)
}

```

```

BestMEMModelRndDF <- BestMEMModelRndDF %>% arrange(AIC)
BestMEMModelRndDF <- rbind(BestMEMModelRndDF,
                           data.frame(Name=c("Random average"),
                                       AIC=c(mean(RndModelAIC))))
BestMEMModelRndDF <- rbind(BestMEMModelRndDF,
                           data.frame(Name=c("Random SD"),
                                       AIC=c(sd(RndModelAIC))))

write.csv(BestMEMModelRndDF,
          paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask,
                 "_best_main_effects_model_with_random_cum_term.csv"),
          row.names = FALSE)
}

syll_component_summary <- PosDat %>%
  group_by(syll_component) %>% summarise(MeanPres = mean(preserved),
                                         N = n())
write.csv(syll_component_summary, paste0(TablesDir, CurPat, "_",
                                         RunLabel, "_", CurTask,
                                         "_syllable_component_summary.csv"), row.names = FALSE)
kable(syll_component_summary)

```

syll_component	MeanPres	N
1	0.9125000	560
O	0.8477491	1977
P	0.7352941	34
S	0.5840000	250
V	0.9345279	1451

```

# main effects models for data without satellite positions

keep_components = c("0", "V", "1")
OV1Data <- PosDat[PosDat$syll_component %in% keep_components,]
OV1Data <- OV1Data %>% select(stim_number,
                             stimlen, stim, pos,
                             preserved, syll_component)
OV1Data$CumPres <- CalcCumPres(OV1Data)
OV1Data$CumErr <- CalcCumErrFromPreserved(OV1Data)

SimpSyllMEAICSummary <- EvaluateSubsetData(OV1Data, MEModelEquations)

## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      1.279      0.490
##
## Degrees of Freedom: 3987 Total (i.e. Null); 3986 Residual
## Null Deviance:      2790

```

```

## Residual Deviance: 2566 AIC: 2570
## log likelihood: -1282.927
## Nagelkerke R2: 0.1086698
## % pres/err predicted correctly: -742.5599
## % of predictable range [ (model-null)/(1-null) ]: 0.05960577
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) I(pos^2) pos
## 0.18506 -0.07839 0.93209
##
## Degrees of Freedom: 3987 Total (i.e. Null); 3985 Residual
## Null Deviance: 2790
## Residual Deviance: 2594 AIC: 2600
## log likelihood: -1296.874
## Nagelkerke R2: 0.09548471
## % pres/err predicted correctly: -744.1512
## % of predictable range [ (model-null)/(1-null) ]: 0.05759319
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) pos
## 0.9939 0.3239
##
## Degrees of Freedom: 3987 Total (i.e. Null); 3986 Residual
## Null Deviance: 2790
## Residual Deviance: 2640 AIC: 2644
## log likelihood: -1319.962
## Nagelkerke R2: 0.07345422
## % pres/err predicted correctly: -756.9626
## % of predictable range [ (model-null)/(1-null) ]: 0.04139038
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumErr
## 2.1431 -0.1569
##
## Degrees of Freedom: 3987 Total (i.e. Null); 3986 Residual
## Null Deviance: 2790
## Residual Deviance: 2785 AIC: 2789
## log likelihood: -1392.444
## Nagelkerke R2: 0.002611477

```

```
## % pres/err predicted correctly: -788.5871
## % of predictable range [ (model-null)/(1-null) ]: 0.001394256
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
## 2.075
##
## Degrees of Freedom: 3987 Total (i.e. Null); 3987 Residual
## Null Deviance: 2790
## Residual Deviance: 2790 AIC: 2792
## log likelihood: -1395.066
## Nagelkerke R2: 0
## % pres/err predicted correctly: -789.6896
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen
## 2.16729 -0.01204
##
## Degrees of Freedom: 3987 Total (i.e. Null); 3986 Residual
## Null Deviance: 2790
## Residual Deviance: 2790 AIC: 2794
## log likelihood: -1394.983
## Nagelkerke R2: 8.28357e-05
## % pres/err predicted correctly: -789.6572
## % of predictable range [ (model-null)/(1-null) ]: 4.092283e-05
## *****
```

```
write.csv(SimpSyllMEAICSummary,
  paste0(TablesDir, CurPat, "_", RunLabel, "_",
    CurTask,
    "_OV1_main_effects_model_summary.csv"),
  row.names = FALSE)
kable(SimpSyllMEAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumPres	2569.853	0.00000	1e+00	0.9999999	0.1086698	18.27925	180.490033	NA	NA	NA	NA
preserved ~ (I(pos^2) + pos)	2599.742	29.89405	3e-07	0.0000000	0.0954847	17.18505	187.587	NA	- 0.0783925	0.9320861	NA
preserved ~ pos	2643.923	74.07011	0e+00	0.0000000	0.0734542	16.99391	171.71	NA	NA	0.3239344	NA
preserved ~ CumErr	2788.882	219.03376	0e+00	0.0000000	0.0026112	13.14305	150.550	NA	- 0.1568906	NA	NA
preserved ~ 1	2792.132	22.27815	0e+00	0.0000000	0.0000000	12.07465	148.48	NA	NA	NA	NA

Model	AIC	DeltaAIC	AICexp	AICwgt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ stimlen	2793.965	224.1119	0e+00	0.0000000	0.0000823	1.672880	NA	NA	NA	NA	- 0.0120383

```
# main effects models for data without satellite or coda positions
# (tradeoff -- takes away more complex segments, in addition to satellites, but
# also reduces data)
```

```
keep_components = c("0","V")
OVDData <- PosDat[PosDat$syll_component %in% keep_components,]
OVDData <- OVDData %>% select(stim_number,
                             stimlen,stim,pos,
                             preserved,syll_component)
OVDData$CumPres <- CalcCumPres(OVDData)
OVDData$CumErr <- CalcCumErrFromPreserved(OVDData)

SimpSyllMEAICSummary2<-EvaluateSubsetData(OVDData,MEModelEquations)
```

```
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      1.291      0.556
##
## Degrees of Freedom: 3427 Total (i.e. Null); 3426 Residual
## Null Deviance:      2454
## Residual Deviance: 2263 AIC: 2267
## log likelihood: -1131.551
## Nagelkerke R2: 0.1058345
## % pres/err predicted correctly: -658.6992
## % of predictable range [ (model-null)/(1-null) ]: 0.05825701
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)      pos
##      0.14075      -0.08503      0.97834
##
## Degrees of Freedom: 3427 Total (i.e. Null); 3425 Residual
## Null Deviance:      2454
## Residual Deviance: 2269 AIC: 2275
## log likelihood: -1134.447
## Nagelkerke R2: 0.1027049
## % pres/err predicted correctly: -655.6459
## % of predictable range [ (model-null)/(1-null) ]: 0.06261568
## *****
```

```

## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      0.9966      0.3146
##
## Degrees of Freedom: 3427 Total (i.e. Null); 3426 Residual
## Null Deviance:      2454
## Residual Deviance: 2320 AIC: 2324
## log likelihood: -1160.014
## Nagelkerke R2: 0.07485099
## % pres/err predicted correctly: -669.3169
## % of predictable range [ (model-null)/(1-null) ]: 0.04309988
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
##      2.036
##
## Degrees of Freedom: 3427 Total (i.e. Null); 3427 Residual
## Null Deviance:      2454
## Residual Deviance: 2454 AIC: 2456
## log likelihood: -1226.886
## Nagelkerke R2: -4.343592e-16
## % pres/err predicted correctly: -699.5088
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.06122      -0.06899
##
## Degrees of Freedom: 3427 Total (i.e. Null); 3426 Residual
## Null Deviance:      2454
## Residual Deviance: 2453 AIC: 2457
## log likelihood: -1226.537
## Nagelkerke R2: 0.0003974047
## % pres/err predicted correctly: -699.362
## % of predictable range [ (model-null)/(1-null) ]: 0.0002094499
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",

```

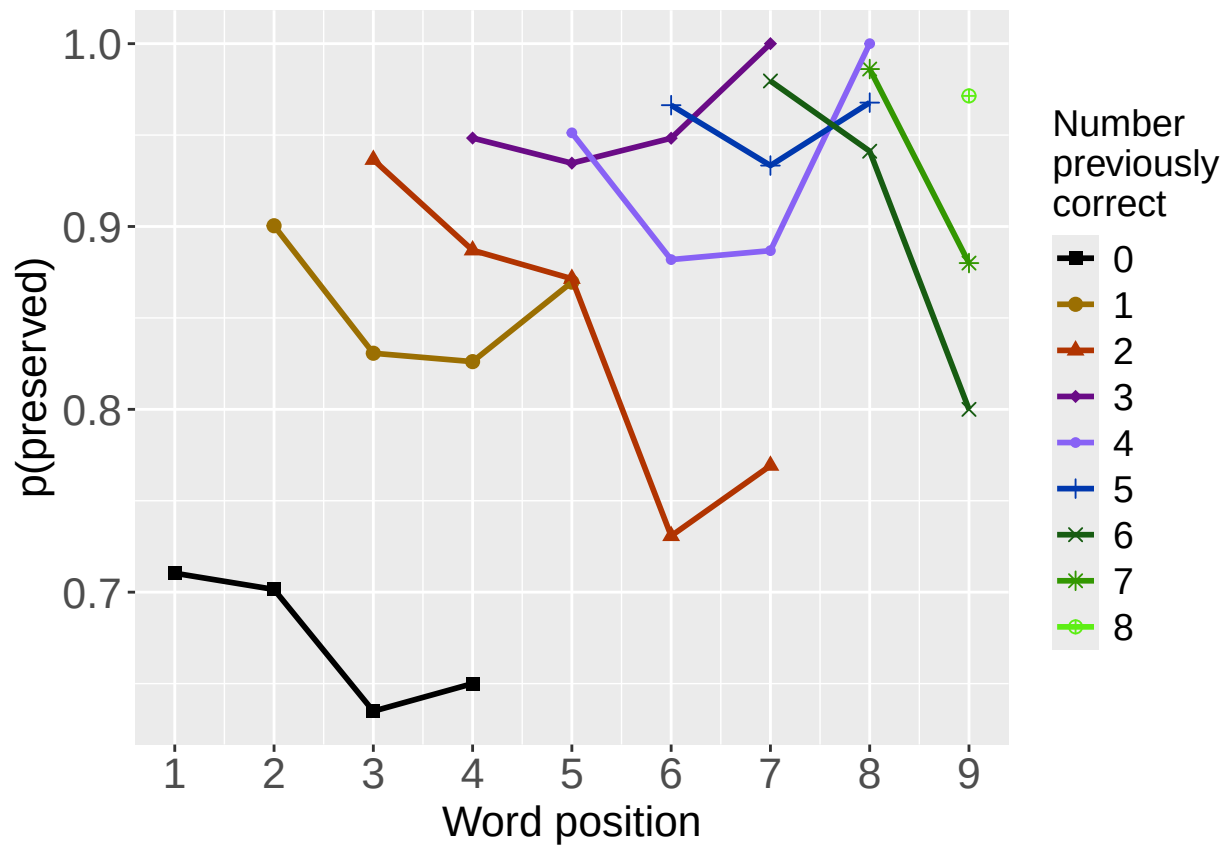
```
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      2.12650      -0.01185
##
## Degrees of Freedom: 3427 Total (i.e. Null);  3426 Residual
## Null Deviance:      2454
## Residual Deviance: 2454  AIC: 2458
## log likelihood:  -1226.814
## Nagelkerke R2:   8.233892e-05
## % pres/err predicted correctly:  -699.4797
## % of predictable range [ (model-null)/(1-null) ]:  4.148864e-05
## *****
```

```
write.csv(SimpSyllMEAICSummary2,
          paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask,
                "_OV_main_effects_model_summary.csv"), row.names = FALSE)
kable(SimpSyllMEAICSummary2)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPre	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumPres	2267.102	0.000000	1.000000	0.980092	0.105834	5.290597	90.5560088	NA	NA	NA	NA
preserved ~ (I(pos^2) + pos)	2274.895	7.793141	0.020311	0.101990	0.102704	4.1407481	NA	NA	- 0.0850277	0.9783401	NA
preserved ~ pos	2324.023	56.925652	0.000000	0.000000	0.074851	0.9966045	NA	NA	NA	0.3145657	NA
preserved ~ 1	2455.771	188.669561	0.000000	0.000000	0.000000	0.0355635	NA	NA	NA	NA	NA
preserved ~ CumErr	2457.073	189.973080	0.000000	0.000000	0.000397	1.0612205	NA	- 0.0689887	NA	NA	NA
preserved ~ stimlen	2457.621	190.525267	0.000000	0.000000	0.000082	1.1265015	NA	NA	NA	NA	- 0.0118454

```
# plot prev err and prev cor plots
PrevCorPlot<-PlotObsPreviousCorrect(PosDat,palette_values,shape_values)
```

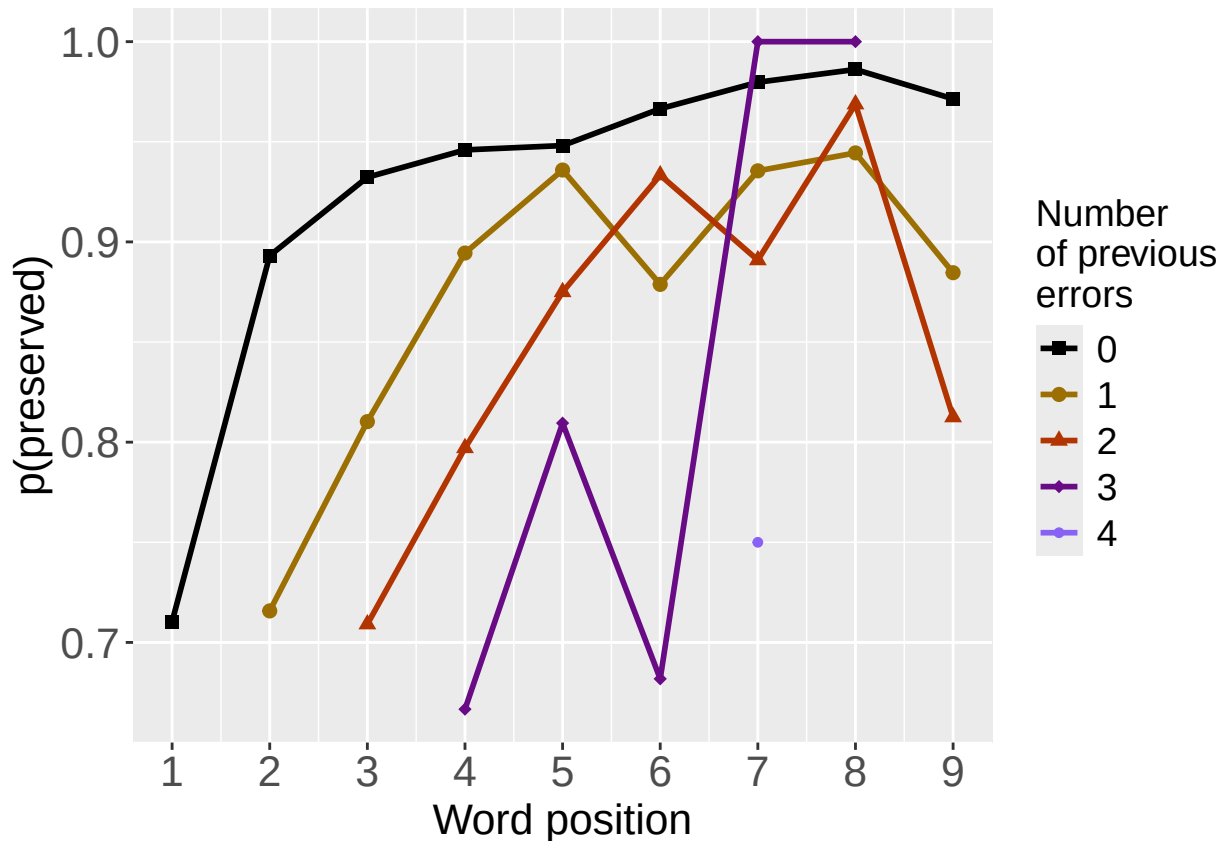
```
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
print(PrevCorPlot)
```



```
PrevErrPlot<-PlotObsPreviousError(PosDat,palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
```

```
print(PrevErrPlot)
```

```
PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_PrevCorPlot_Obs")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevCorPlot,device="tiff",compression = "lzw")

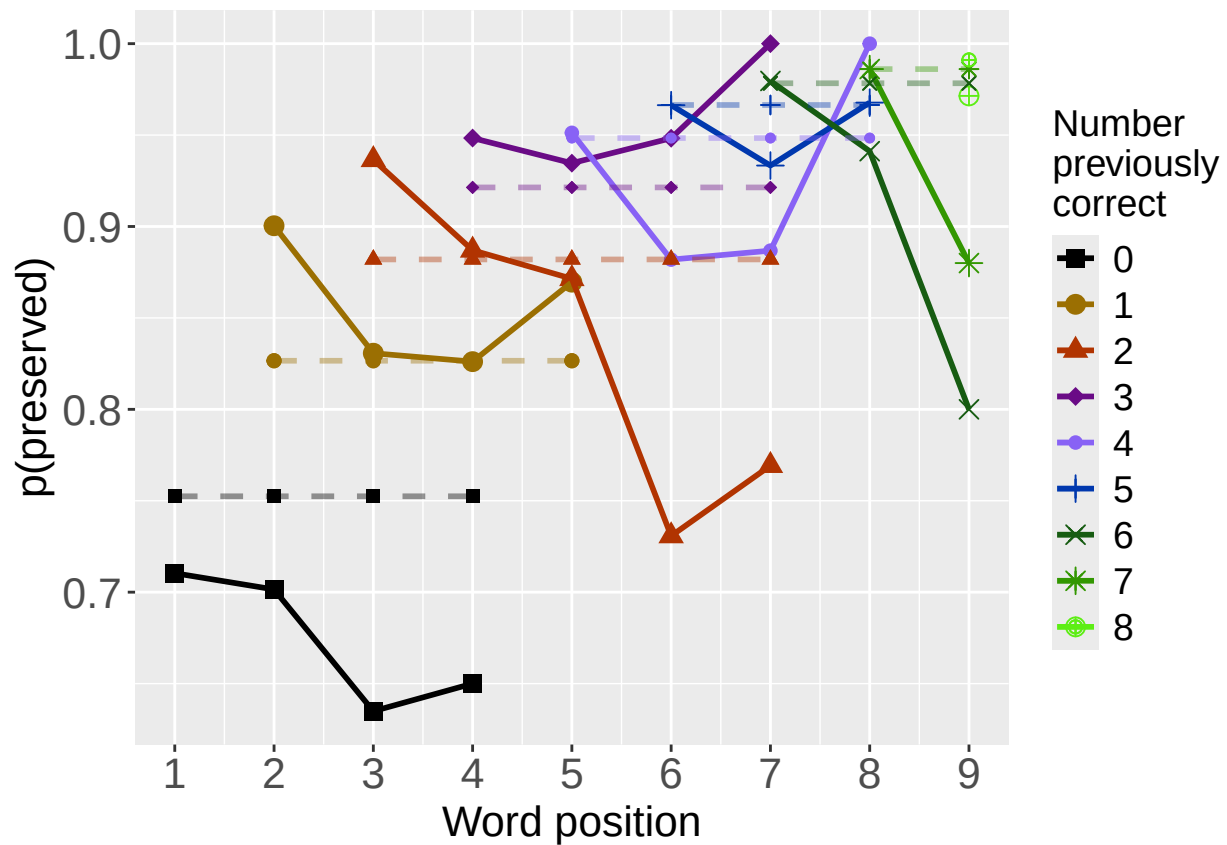
## Saving 6.5 x 4.5 in image

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_PrevErrPlot_Obs")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevErrPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image
# plot prev err and prev cor with predicted values
MEModel<-MERes$ModelResult[[ BestModelIndexL1 ]]
PosDat$MEPred<-fitted(MEModel)

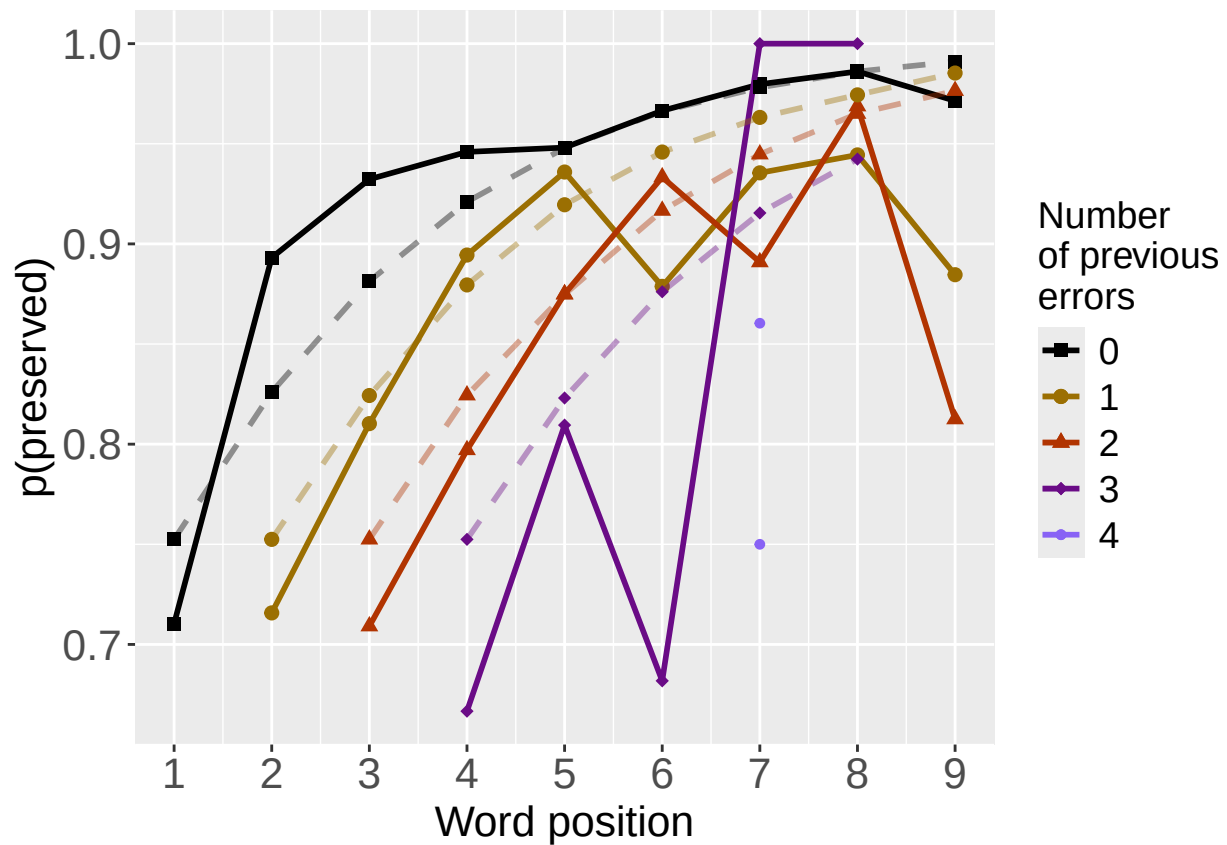
PrevCorPlot<-PlotObsPredPreviousCorrect(PosDat,"preserved","MEPred",palette_values,shape_values)

## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
print(PrevCorPlot)
```



```
PrevErrPlot<-PlotObsPredPreviousError(PosDat,"preserved","MEPred",palette_values,shape_values)

## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
print(PrevErrPlot)
```



```

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",
                  CurTask,"PrevCorPlot_ObsPredME")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevCorPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",
                  CurTask,"PrevErrPlot_ObsPredME")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevErrPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image

```

```

CumAICSummary <- NULL

#####
# level 2 -- Add position squared (quadratic with position)
#####

# After establishing the primary variable, see about additions

BestMEFactor<-BestMEModelFormula
OtherFactor<-"I(pos^2) + pos"
OtherFactorName<-"quadratic_by_pos"

if(!grepl("pos",BestMEFactor,fixed = TRUE)){ # if best model does not include pos
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor,OtherFactorName)
  CumAICSummary <- AICSummary
  kable(AICSummary)
}

```

```

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres      I(pos^2)          pos
##    0.67900      0.62413     -0.06672      0.34545
##
## Degrees of Freedom: 4271 Total (i.e. Null);  4268 Residual
## Null Deviance:      3311
## Residual Deviance: 3006  AIC: 3014
## log likelihood:  -1503.226
## Nagelkerke R2:  0.1277004
## % pres/err predicted correctly:  -897.8932
## % of predictable range [ (model-null)/(1-null) ]:  0.07352594
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",

```

```

##      data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      1.1118      0.4498
##
## Degrees of Freedom: 4271 Total (i.e. Null);  4270 Residual
## Null Deviance:      3311
## Residual Deviance: 3055  AIC: 3059
## log likelihood:  -1527.326
## Nagelkerke R2:  0.1081112
## % pres/err predicted correctly:  -909.2099
## % of predictable range [ (model-null)/(1-null) ]:  0.06186207
## *****
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)      pos
##      0.24058      -0.05991      0.76966
##
## Degrees of Freedom: 4271 Total (i.e. Null);  4269 Residual
## Null Deviance:      3311
## Residual Deviance: 3115  AIC: 3121
## log likelihood:  -1557.441
## Nagelkerke R2:  0.08332146
## % pres/err predicted correctly:  -920.1325
## % of predictable range [ (model-null)/(1-null) ]:  0.05060432
## *****

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	I(pos^2)	pos
preserved ~ CumPres + I(pos^2) + pos	3014.452	0.00000	1	1	0.1277004	0.6789994	0.6241268	-0.0667186	0.3454509
preserved ~ CumPres	3058.653	44.20132	0	0	0.1081112	1.1117700	0.4497816	NA	NA
preserved ~ I(pos^2) + pos	3120.881	106.42979	0	0	0.0833215	0.2405752	NA	-0.0599137	0.7696604

```
#####
# level 2 -- Add length
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"stimlen"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumPres stimlen
## 2.2006 0.4783 -0.1490
##
## Degrees of Freedom: 4271 Total (i.e. Null); 4269 Residual
## Null Deviance: 3311
## Residual Deviance: 3026 AIC: 3032
## log likelihood: -1512.773
## Nagelkerke R2: 0.1199671
## % pres/err predicted correctly: -900.145
## % of predictable range [ (model-null)/(1-null) ]: 0.07120507
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumPres
## 1.1118 0.4498
##
## Degrees of Freedom: 4271 Total (i.e. Null); 4270 Residual
## Null Deviance: 3311
## Residual Deviance: 3055 AIC: 3059
## log likelihood: -1527.326
## Nagelkerke R2: 0.1081112
## % pres/err predicted correctly: -909.2099
## % of predictable range [ (model-null)/(1-null) ]: 0.06186207
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
```

```
## (Intercept)      stimlen
##      2.10960      -0.02778
##
## Degrees of Freedom: 4271 Total (i.e. Null);  4270 Residual
## Null Deviance:      3311
## Residual Deviance: 3310  AIC: 3314
## log likelihood:  -1655.117
## Nagelkerke R2:  0.0004673632
## % pres/err predicted correctly:  -968.9911
## % of predictable range [ (model-null)/(1-null) ]:  0.0002466108
## *****
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	stimlen
preserved ~ CumPres + stimlen	3031.545	0.00000	1.0e+00	0.9999987	0.1199671	2.200571	0.4782575	-0.1490341
preserved ~ CumPres	3058.653	27.10773	1.3e-06	0.0000013	0.1081112	1.111770	0.4497816	NA
preserved ~ stimlen	3314.234	282.68845	0.0e+00	0.0000000	0.0004674	2.109605	NA	-0.0277784

```
#####
# level 2 -- add cumulative preserved
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"CumPres"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}

#####
# level 2 -- Add linear position (NOT quadratic)
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"pos"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}

## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres          pos
```

```

##      1.3684      0.6206      -0.1662
##
## Degrees of Freedom: 4271 Total (i.e. Null);  4269 Residual
## Null Deviance:      3311
## Residual Deviance: 3043  AIC: 3049
## log likelihood:  -1521.564
## Nagelkerke R2:  0.1128152
## % pres/err predicted correctly:  -907.1807
## % of predictable range [ (model-null)/(1-null) ]:  0.06395351
## *****
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      1.1118      0.4498
##
## Degrees of Freedom: 4271 Total (i.e. Null);  4270 Residual
## Null Deviance:      3311
## Residual Deviance: 3055  AIC: 3059
## log likelihood:  -1527.326
## Nagelkerke R2:  0.1081112
## % pres/err predicted correctly:  -909.2099
## % of predictable range [ (model-null)/(1-null) ]:  0.06186207
## *****
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      pos
##      0.8728      0.3039
##
## Degrees of Freedom: 4271 Total (i.e. Null);  4270 Residual
## Null Deviance:      3311
## Residual Deviance: 3147  AIC: 3151
## log likelihood:  -1573.319
## Nagelkerke R2:  0.07010906
## % pres/err predicted correctly:  -929.8403
## % of predictable range [ (model-null)/(1-null) ]:  0.0405986
## *****

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	pos
preserved ~ CumPres + pos	3049.128	0.000000	1.0000000	0.9915289	0.1128152	1.3683867	0.6205913	- 0.1662494
preserved ~ CumPres	3058.653	9.525181	0.0085435	0.0084711	0.1081112	1.1117700	0.4497816	NA
preserved ~ pos	3150.638	101.510593	0.0000000	0.0000000	0.0701091	0.8728195	NA	0.3038868


```

CumAICSummary <- CumAICSummary %>% arrange(AIC)
BestModelFormulaL2 <- CumAICSummary$Model[BestModelIndexL2]
write.csv(CumAICSummary, paste0(TablesDir, CurPat, "_",
                                RunLabel, "_", CurTask,
                                "_main_effects_plus_one_model_summary.csv"), row.names = FALSE)
kable(CumAICSummary)

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPred	(pos^2)	pos	stimlen
preserved ~ CumPres + I(pos^2) + pos	3014.452	0.000000	1.000000	0.000000	0.127700	4.678999	40.6241268	-	0.3454509	NA
preserved ~ CumPres + stimlen	3031.545	0.000000	1.000000	0.999998	0.119967	2.200570	80.4782575	NA	NA	-
preserved ~ CumPres + pos	3049.128	0.000000	1.000000	0.991528	0.112815	2.368386	70.6205913	NA	-	NA
preserved ~ CumPres	3058.653	4.201317	0.000000	0.000000	0.108111	2.111770	0.4497816	NA	NA	NA
preserved ~ CumPres	3058.653	7.107730	0.000000	0.000000	0.108111	2.111770	0.4497816	NA	NA	NA
preserved ~ CumPres	3058.653	9.525181	0.008543	0.008471	0.108111	2.111770	0.4497816	NA	NA	NA
preserved ~ I(pos^2) + pos	3120.881	106.429786	0.000000	0.000000	0.083321	6.2405752	NA	-	0.7696604	NA
preserved ~ pos	3150.638	101.510598	0.000000	0.000000	0.070109	0.8728195	NA	NA	0.3038868	NA
preserved ~ stimlen	3314.234	282.688452	0.000000	0.000000	0.000467	2.1096046	NA	NA	NA	-
										0.0277784

```

# explore influence of frequency and length

if(grepl("stimlen", BestModelFormulaL2)){ # if length is already in the formula
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2, " + log_freq")
  )
}else if(grepl("log_freq", BestModelFormulaL2)){ # if log_freq is already in the formula
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2, " + stimlen")
  )
}else{
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2, " + log_freq"),
    paste0(BestModelFormulaL2, " + stimlen"),
    paste0(BestModelFormulaL2, " + stimlen + log_freq")
  )
}

Level3Res<-TestModels(Level3ModelEquations, PosDat)

```

```

## *****
## model index: 5
##

```

```

## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres      I(pos^2)          pos      stimlen      log_freq
##      1.32556      0.60279     -0.06267      0.36187     -0.09365      0.08777
##
## Degrees of Freedom: 4271 Total (i.e. Null);  4266 Residual
## Null Deviance:      3311
## Residual Deviance: 2978 AIC: 2990
## log likelihood: -1488.877
## Nagelkerke R2:  0.1392588
## % pres/err predicted correctly: -889.5231
## % of predictable range [ (model-null)/(1-null) ]:  0.08215286
## *****
## model index:  3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres      I(pos^2)          pos      log_freq
##      0.65185      0.61711     -0.06659      0.35996      0.10892
##
## Degrees of Freedom: 4271 Total (i.e. Null);  4267 Residual
## Null Deviance:      3311
## Residual Deviance: 2987 AIC: 2997
## log likelihood: -1493.691
## Nagelkerke R2:  0.1353896
## % pres/err predicted correctly: -893.4032
## % of predictable range [ (model-null)/(1-null) ]:  0.07815377
## *****
## model index:  4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres      I(pos^2)          pos      stimlen
##      1.53465      0.60251     -0.06158      0.35255     -0.12019
##
## Degrees of Freedom: 4271 Total (i.e. Null);  4267 Residual
## Null Deviance:      3311
## Residual Deviance: 2989 AIC: 2999
## log likelihood: -1494.623
## Nagelkerke R2:  0.1346394
## % pres/err predicted correctly: -891.4643
## % of predictable range [ (model-null)/(1-null) ]:  0.08015211
## *****
## model index:  1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##

```

```

## Coefficients:
## (Intercept)      CumPres      I(pos^2)      pos
##      0.67900      0.62413     -0.06672      0.34545
##
## Degrees of Freedom: 4271 Total (i.e. Null);  4268 Residual
## Null Deviance:      3311
## Residual Deviance: 3006  AIC: 3014
## log likelihood:  -1503.226
## Nagelkerke R2:  0.1277004
## % pres/err predicted correctly:  -897.8932
## % of predictable range [ (model-null)/(1-null) ]:  0.07352594
## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)
##      1.896
##
## Degrees of Freedom: 4271 Total (i.e. Null);  4271 Residual
## Null Deviance:      3311
## Residual Deviance: 3311  AIC: 3313
## log likelihood:  -1655.655
## Nagelkerke R2:  0
## % pres/err predicted correctly:  -969.2303
## % of predictable range [ (model-null)/(1-null) ]:  0
## *****

BestModelL3<-Level3Res$ModelResult[[ BestModelIndexL3 ]]
BestModelFormulaL3<-Level3Res$Model[[BestModelIndexL3]]

AICSummary<-data.frame(Model=Level3Res$Model,
                        AIC=Level3Res$AIC,
                        row.names = Level3Res$Model)
AICSummary$DeltaAIC<-AICSummary$AIC-AICSummary$AIC[1]
AICSummary$AICexp<-exp(-0.5*AICSummary$DeltaAIC)
AICSummary$AICwt<-AICSummary$AICexp/sum(AICSummary$AICexp)
AICSummary$NagR2<-Level3Res$NagR2

AICSummary <- merge(AICSummary,Level3Res$CoefficientValues,
                    by='row.names',sort=FALSE)
AICSummary <- subset(AICSummary, select = -c(Row.names))

write.csv(AICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                           CurTask,"_main_effects_plus_one_len_freq_summary.csv"),
          row.names = FALSE)

kable(AICSummary)

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	I(pos^2)	pos	log_freq	stimlen
preserved ~ CumPres + I(pos^2) + pos + stimlen + log_freq	2989.751	0.000000	0.000000	0.070169	0.139251	0.825559	0.602789	-	0.361866	0.087768	-
								0.062673			0.093646
preserved ~ CumPres + I(pos^2) + pos + log_freq	2997.382	0.628105	0.022058	0.021400	0.135389	0.651854	0.561711	-	0.359960	0.108924	NA
								0.066591			
preserved ~ CumPres + I(pos^2) + pos + stimlen	2999.246	0.492458	0.008684	0.008425	0.134639	0.534648	0.602514	-	0.352554	NA	-
								0.061576			0.120188
preserved ~ CumPres + I(pos^2) + pos	3014.452	0.698047	0.000000	0.000000	0.127700	0.478999	0.624126	-	0.345450	NA	NA
								0.066718			
preserved ~ 1	3313.332	0.557078	0.000000	0.000000	0.000000	0.895505	NA	NA	NA	NA	NA

```

# BestModelIndex is set by the parameter file and is used to choose
# a model that is essentially equivalent to the lowest AIC model, but
# simpler (and with a slightly higher AIC value, so not in first position)
BestModelL3<-Level3Res$ModelResult[[ BestModelIndexL3 ]]
BestModelFormulaL3<-Level3Res$Model[BestModelIndexL3]

BestModel<-BestModelL3
BestModelDeletions<-arrange(dropterm(BestModel),desc(AIC))
# order by terms that increase AIC the most when they are the one dropped
BestModelDeletions

## Single term deletions
##
## Model:
## preserved ~ CumPres + I(pos^2) + pos + stimlen + log_freq
##           Df Deviance   AIC
## CumPres   1   3077.7 3087.7
## I(pos^2)   1   3009.8 3019.8
## pos        1   2992.1 3002.1
## log_freq   1   2989.2 2999.2
## stimlen    1   2987.4 2997.4
## <none>      1   2977.8 2989.8

#####
# Single deletions from best model
#####

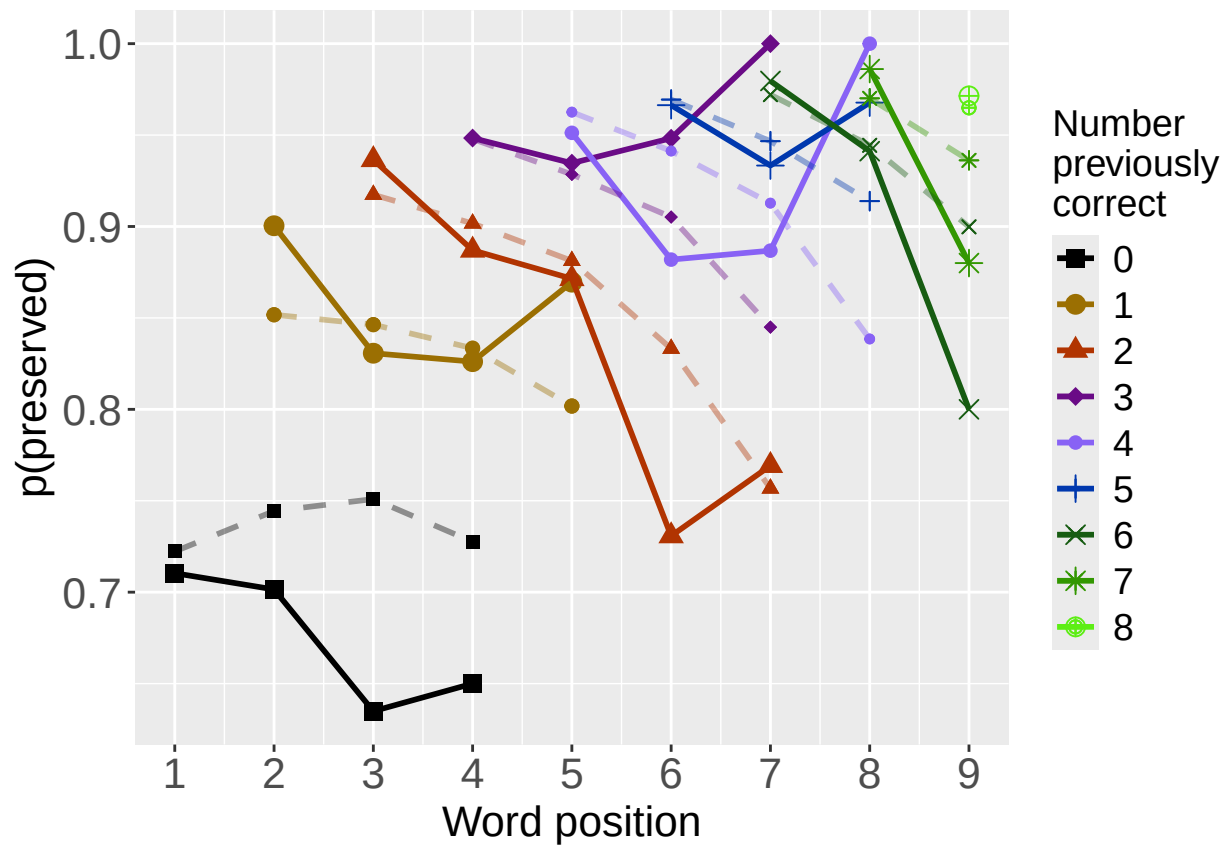
write.csv(BestModelDeletions,paste0(TablesDir,CurPat,"_",
                                   RunLabel,"_",CurTask,
                                   "_best_model_single_term_deletions.csv"),
          row.names = TRUE)

# plot prev err and prev cor with predicted values
PosDat$OAPred<-fitted(BestModel)

PrevCorPlot<-PlotObsPredPreviousCorrect(PosDat,"preserved","OAPred",palette_values,shape_values)

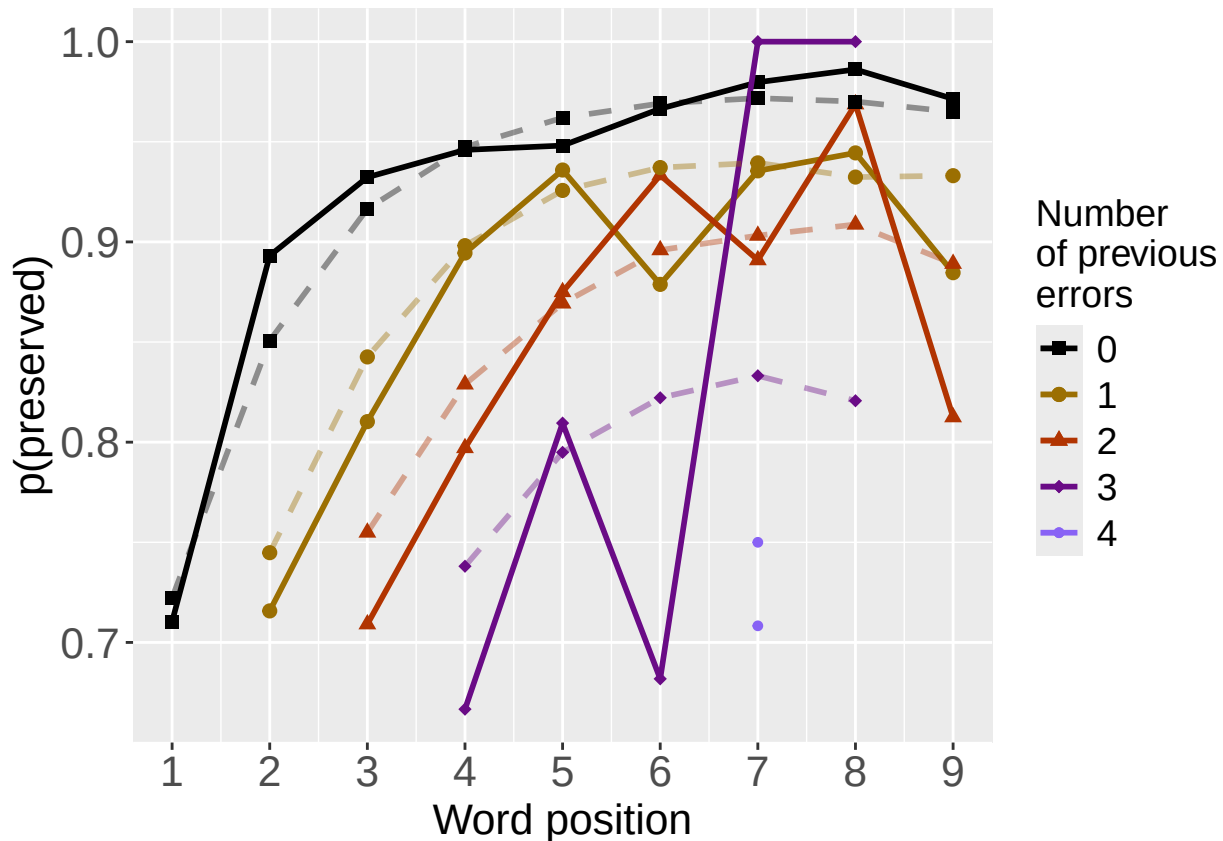
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
print(PrevCorPlot)

```



```
PrevErrPlot<-PlotObsPredPreviousError(PosDat,"preserved","OAPred",palette_values,shape_values)

## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
print(PrevErrPlot)
```



```

PlotName<-paste0(FigDir,CurPat,"_",
  RunLabel,"_",CurTask,
  "_ObsPred_BestFullModel")
ggsave(filename=paste(PlotName,"_prev_correct.tif",sep=""),plot=PrevCorPlot,device="tiff",compression =

## Saving 6.5 x 4.5 in image
ggsave(filename=paste(PlotName,"_prev_error.tif",sep=""),plot=PrevErrPlot,device="tiff",compression = "

## Saving 6.5 x 4.5 in image
if(DoSimulations){
  BestModelFormulaL3Rnd <- BestModelFormulaL3
  if(grepl("CumPres",BestModelFormulaL3Rnd)){
    BestModelFormulaL3Rnd <- gsub("CumPres","RndCumPres",BestModelFormulaL3Rnd)
  }
  if(grepl("CumErr",BestModelFormulaL3Rnd)){
    BestModelFormulaL3Rnd <- gsub("CumErr","RndCumErr",BestModelFormulaL3Rnd)
  }

  RndModelAIC<-numeric(length=RandomSamples)
  for(rindex in seq(1,RandomSamples)){
    # Shuffle cumulative values
    PosDat$RndCumPres <- ShuffleWithinWord(PosDat,"CumPres")
    PosDat$RndCumErr <- ShuffleWithinWord(PosDat,"CumErr")
    BestModelRnd <- glm(as.formula(BestModelFormulaL3Rnd),
      family="binomial",data=PosDat)
    RndModelAIC[rindex] <- BestModelRnd$aic
  }
}

```

```

}
ModelNames<-c(paste0("***",BestModelFormulaL3),
              rep(BestModelFormulaL3Rnd,RandomSamples))
AICValues <- c(BestModelL3$aic,RndModelAIC)
BestModelRndDF <- data.frame(Name=ModelNames,AIC=AICValues)
BestModelRndDF <- BestModelRndDF %>% arrange(AIC)
BestModelRndDF <- rbind(BestModelRndDF,
                        data.frame(Name=c("Random average"),
                                   AIC=c(mean(RndModelAIC))))
BestModelRndDF <- rbind(BestModelRndDF,
                        data.frame(Name=c("Random SD"),
                                   AIC=c(sd(RndModelAIC))))
write.csv(BestModelRndDF,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                                "_best_model_with_random_cum_term.csv"),
          row.names = FALSE)
}

# plot influence factors
num_models <- nrow(BestModelDeletions) - 1
# minus 1 because last
# model is always <none> and we don't want to include that

if(num_models > 4){
  last_model_num <- 4
}else{
  last_model_num <- num_models
}

FinalModelSet<-ModelSetFromDeletions(BestModelFormulaL3,
                                     BestModelDeletions,
                                     last_model_num)

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_FactorPlots")

FactorPlot<-PlotModelSetWithFreq(PosDat,FinalModelSet,
                                 palette_values,FinalModelSet,PptID=CurPat)

## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      1.1118      0.4498
##
## Degrees of Freedom: 4271 Total (i.e. Null); 4270 Residual
## Null Deviance:      3311
## Residual Deviance: 3055 AIC: 3059
## log likelihood: -1527.326
## Nagelkerke R2: 0.1081112
## % pres/err predicted correctly: -909.2099
## % of predictable range [ (model-null)/(1-null) ]: 0.06186207

```

```

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres      I(pos^2)
##      1.16389      0.70432     -0.03249
##
## Degrees of Freedom: 4271 Total (i.e. Null); 4269 Residual
## Null Deviance:      3311
## Residual Deviance: 3020 AIC: 3026
## log likelihood: -1509.788
## Nagelkerke R2: 0.1223886
## % pres/err predicted correctly: -902.0467
## % of predictable range [ (model-null)/(1-null) ]: 0.06924499
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres      I(pos^2)      pos
##      0.67900      0.62413     -0.06672      0.34545
##
## Degrees of Freedom: 4271 Total (i.e. Null); 4268 Residual
## Null Deviance:      3311
## Residual Deviance: 3006 AIC: 3014
## log likelihood: -1503.226
## Nagelkerke R2: 0.1277004
## % pres/err predicted correctly: -897.8932
## % of predictable range [ (model-null)/(1-null) ]: 0.07352594
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres      I(pos^2)      pos      log_freq
##      0.65185      0.61711     -0.06659      0.35996      0.10892
##
## Degrees of Freedom: 4271 Total (i.e. Null); 4267 Residual
## Null Deviance:      3311
## Residual Deviance: 2987 AIC: 2997
## log likelihood: -1493.691
## Nagelkerke R2: 0.1353896
## % pres/err predicted correctly: -893.4032
## % of predictable range [ (model-null)/(1-null) ]: 0.07815377
## *****
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

```



```
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes o
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`)

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes o
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 6 rows containing missing values or values outside the scale range (`geom_point()`)

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes o
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes o
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 6 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 6 rows containing missing values or values outside the scale range (`geom_point()`).

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes o
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).

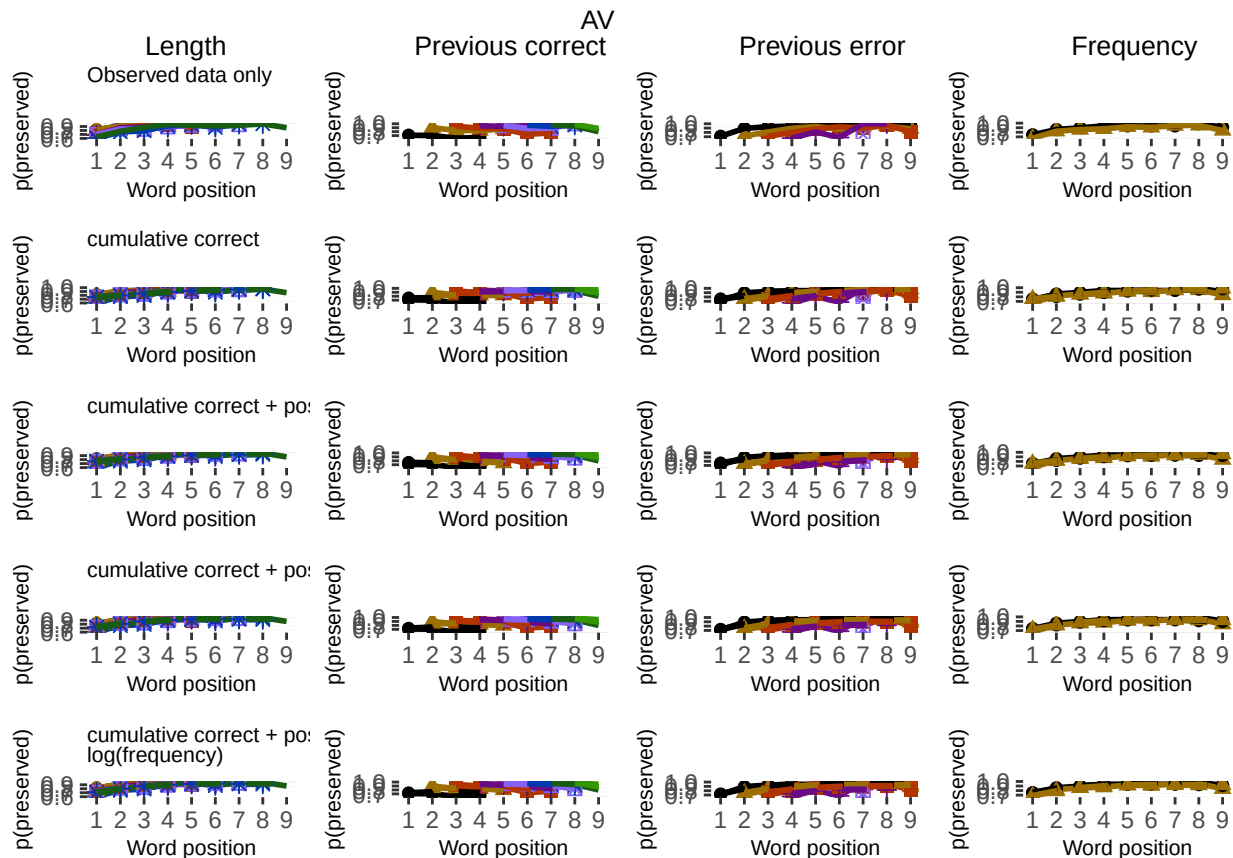
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes o
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 6 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 6 rows containing missing values or values outside the scale range (`geom_point()`).

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes o
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
```

```
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 6 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 6 rows containing missing values or values outside the scale range (`geom_point()`)
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes c
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`)
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes c
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 6 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 6 rows containing missing values or values outside the scale range (`geom_point()`)
ggsave(paste0(PlotName, ".tif"), plot=FactorPlot, width = 360, height=400, units="mm", device="tiff", compress=
# use \blandscape and \elandscape to make markdown plots landscape if needed
FactorPlot
```



```
DA.Result<-dominanceAnalysis(BestModel)
DAContributionAverage<-ConvertDominanceResultToTable(DA.Result)
write.csv(DAContributionAverage,paste0(TablesDir,CurPat,"_",RunLabel,"_",
CurTask,"_dominance_analysis_table.csv"),
row.names = TRUE)
kable(DAContributionAverage)
```

	CumPres	I(pos^2)	pos	stimlen	log_freq
McFadden	0.0501301	0.0190098	0.0231548	0.0040558	0.0043821

	CumPres	I(pos^2)	pos	stimlen	log_freq
SquaredCorrelation	0.0371810	0.0142751	0.0174423	0.0029710	0.0032400
Nagelkerke	0.0371810	0.0142751	0.0174423	0.0029710	0.0032400
Estrella	0.0393784	0.0148777	0.0181024	0.0031981	0.0034461

	deviance	deviance_explained
CumPres + I(pos ²) + pos + log_freq	2987.382	323.9290
CumPres + I(pos ²) + pos	3006.452	304.8590
CumPres + I(pos ²)	3019.576	291.7349
CumPres	3054.653	256.6577
null	3311.311	0.0000

```
deviance_differences_df<-GetDevianceSet(FinalModelSet,PosDat)
write.csv(deviance_differences_df,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                                         CurTask,"_deviance_differences_table.csv"),
          row.names = FALSE)
deviance_differences_df
```

```
##                                model deviance deviance_explained percent_explained
## CumPres + I(pos^2) + pos + log_freq CumPres + I(pos^2) + pos + log_freq 2987.382      323.9290      9.782500
## CumPres + I(pos^2) + pos              CumPres + I(pos^2) + pos 3006.452      304.8590      9.206597
## CumPres + I(pos^2)                    CumPres + I(pos^2) 3019.576      291.7349      8.810254
## CumPres                                CumPres 3054.653      256.6577      7.750940
## null                                  null 3311.311      0.0000      0.000000
##                                percent_of_explained_deviance increment_in_explained
## CumPres + I(pos^2) + pos + log_freq      100.00000      5.887075
## CumPres + I(pos^2) + pos                  94.11292      4.051552
## CumPres + I(pos^2)                       90.06137     10.828660
## CumPres                                  79.23271     79.232713
## null                                     NA      0.000000
```

```
kable(deviance_differences_df[,2:3], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
kable(deviance_differences_df[,c(4:6)], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
NagContributions <- t(DAContributionAverage[3,])
NagTotal<-sum(NagContributions)
NagPercents <- NagContributions / NagTotal
NagResultTable <- data.frame(NagContributions = NagContributions, NagPercents = NagPercents)
names(NagResultTable)<-c("NagContributions","NagPercents")
write.csv(NagResultTable,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
```

	percent_explained	percent_of_explained_deviance	increment_in_explained
CumPres + I(pos ²) + pos + log_freq	9.782500	100.00000	5.887075
CumPres + I(pos ²) + pos	9.206597	94.11292	4.051552
CumPres + I(pos ²)	8.810254	90.06137	10.828660
CumPres	7.750940	79.23271	79.232713
null	0.000000	NA	0.000000

```

                                "_nagelkerke_contributions_table.csv"),row.names = TRUE)
NagPercents

```

```

##           Nagelkerke
## CumPres  0.49502387
## I(pos^2) 0.19005725
## pos      0.23222528
## stimlen  0.03955628
## log_freq 0.04313731

```

```

sse_results_list<-compare_SS_accounted_for(FinalModelSet,"preserved ~ 1",PosDat,N_cutoff=10)

```

```

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

```

```

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length

sse_table<-sse_results_table(sse_results_list)
write.csv(sse_table,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                           CurTask,"_sse_results_table.csv"),row.names = TRUE)

sse_table

##               model p_accounted_for model_deviance diff_CumPres diff_CumPres+I(pos^2) diff_CumPres+I(pos^2)+pos
## 1      preserved ~ CumPres      0.7460177      3054.653      0.00000000      -0.05486495      -0.07666197
## 2      preserved ~ CumPres+I(pos^2) 0.8008826      3019.576      0.05486495      0.00000000      -0.02179703
## 3      preserved ~ CumPres+I(pos^2)+pos 0.8226797      3006.452      0.07666197      0.02179703      0.00000000
## 4 preserved ~ CumPres+I(pos^2)+pos+log_freq 0.8371456      2987.382      0.09112790      0.03626295      0.01446593
## diff_CumPres+I(pos^2)+pos+log_freq
## 1      -0.09112790
## 2      -0.03626295
## 3      -0.01446593
## 4      0.00000000

```

model	p_accounted_for	model_deviance
preserved ~ CumPres	0.7460177	3054.653
preserved ~ CumPres+I(pos ²)	0.8008826	3019.576
preserved ~ CumPres+I(pos ²)+pos	0.8226797	3006.452
preserved ~ CumPres+I(pos ²)+pos+log_freq	0.8371456	2987.382

model	diff_CumPres	diff_CumPres+I(pos ²)	diff_CumPres+I(pos ²)+pos	diff_CumPres+I(pos ²)+pos+log_freq
preserved ~ CumPres	0.0000000	-0.0548649	-0.0766620	-0.0911279
preserved ~ CumPres+I(pos ²)	0.0548649	0.0000000	-0.0217970	-0.0362630
preserved ~ CumPres+I(pos ²)+pos	0.0766620	0.0217970	0.0000000	-0.0144659
preserved ~ CumPres+I(pos ²)+pos+log_freq	0.0911279	0.0362630	0.0144659	0.0000000

```
kable(sse_table[,1:3], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
write.csv(results_report_DF, paste0(TablesDir, CurPat, "_", RunLabel, "_",
  CurTask, "_results_report_df.csv"), row.names = FALSE)
```

```
ncols_in_table <- ncol(sse_table)
kable(sse_table[,c(1,4:ncols_in_table)], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```